창의융합종합설계 최종 결과보고서

<table>
<tr><td>작품 명칭</td><td colspan="5">Sweet & Sweat</td></tr>
<tr><td>개발기간</td><td colspan="5">2026년 3월 4일 ~ 2026년 6월 10일</td></tr>
<tr><td rowspan="4">책 임 자</td><td>성 명</td><td colspan="4">하대영</td></tr>
<tr><td>학 년</td><td colspan="2">4</td><td>학 번</td><td>20231221</td></tr>
<tr><td rowspan="2">연락처</td><td>E-mail</td><td colspan="3">4pril17@kumoh.ac.kr</td></tr>
<tr><td>휴대전화</td><td colspan="3">010-4313-9766</td></tr>
<tr><td rowspan="5">참여학생</td><td colspan="2">학번</td><td>이름</td><td colspan="2">연락처 (휴대전화)</td></tr>
<tr><td colspan="2">20230377</td><td>노가영</td><td colspan="2">010-2308-9268</td></tr>
<tr><td colspan="2">20231246</td><td>함서현</td><td colspan="2">010-5754-0492</td></tr>
<tr><td colspan="2"></td><td></td><td colspan="2"></td></tr>
<tr><td colspan="2"></td><td></td><td colspan="2"></td></tr>
<tr><td colspan="6">본인은 소프트웨어공학심화프로그램 종합설계에 대한 최종보고서를 첨부와 같이 제출합니다.

첨부 : 창의융합종합설계 최종보고서

2026년 6월 3일

책임자 하대영 (인)

소프트웨어공학심화프로그램 PD 귀하</td></tr>
</table>

< 목 차 >

1. 개요 ····· 1
1.1 문제 정의 ····· 1
1.2 목표 ····· 1

2. 요구사항 및 기능명세 ····· 2

3. 시스템 구조도 ····· 45

4. 시스템 설계도 ····· 45
(flowchart, DTD, 회로도 등등)

5. H/W 또는 시스템 구성 ····· 48

6. 구현 기술 ····· 49

7. 주요 자료 구조 또는 DB 구조 ····· 51

8. 주요 함수 명세 ····· 53

9. 설치 매뉴얼 및 사용자 매뉴얼 ····· 66

10. 역할 분담 ····· 80

11. 비고 (감상 포함) ····· 80

12. 회의록 (단계별 최소 2회 이상 - 제안서/요구/설계/구현) … 8

13. 팀원별 포트폴리오 (개인별 개발 사항 설명. 팀원별 1쪽) ‥ 97

1. 개요

1.1 문제 정의

최근 운동 관리 서비스는 기록 저장과 통계 제공에 집중하는 경우가 많아 사용자가 운동을 지속하도록 유도하는 상호작용 요소가 부족하다. Sweet & Sweat은 이러한 한계를 보완하기 위해 AI 캐릭터 챗, 운동 퀘스트, 보상, 캐릭터 성장, 경쟁 요소를 하나의 흐름으로 결합한 게이미피케이션 기반 운동 서비스로 설계하였다.

본 프로젝트의 목적은 사용자가 직접 루틴을 설정하고 컨디션을 입력한 뒤, 캐릭터와의 상호작용 안에서 오늘의 퀘스트를 수행하며 보상과 경쟁을 경험하도록 하는 것이다. 이를 통해 운동 수행을 단순한 기록 행위가 아니라 반복 참여가 가능한 서비스 경험으로 전환하고자 한다.

1.2 목표

본 시스템의 목표는 단순 운동 기록 서비스를 넘어서, 사용자가 운동을 지속하도록 동기를 부여하는 상호작용 중심의 운동 습관 형성 플랫폼을 구현하는 데 있다. 이를 위해 사용자가 루틴을 설정하고 퀘스트를 수행하며 보상을 획득하고 캐릭터 성장을 경험하는 일련의 흐름을 자연스럽게 연결하여 운동을 반복하게 만드는 서비스 구조를 제공한다.

또한 사용자 기능, 루틴 관리, 퀘스트 수행, 캐릭터 및 경제 시스템, 경쟁 기능을 유기적으로 연계하여 서비스의 완성도를 높이고자 한다. 최종적으로는 AI 캐릭터 챗 기반 인터페이스와 게임적 보상 체계를 통해 사용자 참여도와 운동 지속성을 향상시키는 것을 목표로 한다.

2. 요구사항 및 기능명세

2.1 기능리스트

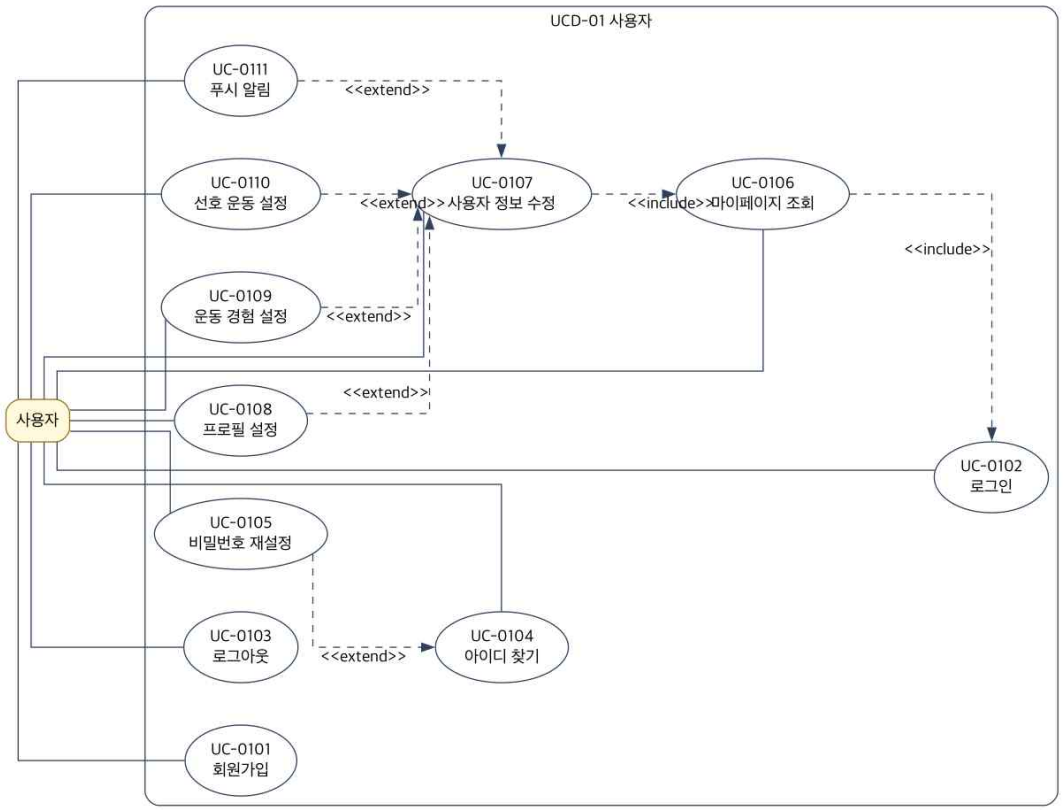
유즈케이스 다이어그램 명	유즈케이스 다이어그램 식별자	유즈케이스 명	유즈케이스 식별자
사용자	UCD-01	회원가입	UC-0101
		로그인	UC-0102
		로그아웃	UC-0103
		아이디 찾기	UC-0104
		비밀번호 재설정	UC-0105
		마이페이지 조회	UC-0106

		사용자 정보 수정	UC-0107
		프로필 설정	UC-0108
		운동 경험 설정	UC-0109
		선호 운동 설정	UC-0110
		푸시 알람 전송	UC-0111
루틴	UCD-02	디폴트 루틴 조회	UC-0201
		루틴 상세 조회	UC-0202
		사용자 루틴 생성	UC-0203
		사용자 루틴 수정	UC-0204
		사용자 루틴 삭제	UC-0205
		컨디션 입력	UC-0206
퀘스트	UCD-03	오늘의 퀘스트 조회	UC-0301
		퀘스트 생성	UC-0302
		퀘스트 수행 기록	UC-0303
		퀘스트 완료 처리	UC-0304
		퀘스트 이력 조회	UC-0305
스토리	UCD-04	캐릭터와 대화	UC-0401
		스토리 진행	UC-0402
		스토리 상태 조회	UC-0403
		스토리 이벤트 발생	UC-0404
캐릭터	UCD-05	캐릭터 조회	UC-0501
		캐릭터 설정	UC-0502
		보유 캐릭터/요소 조회	UC-0503
경제	UCD-06	보상 지급	UC-0601

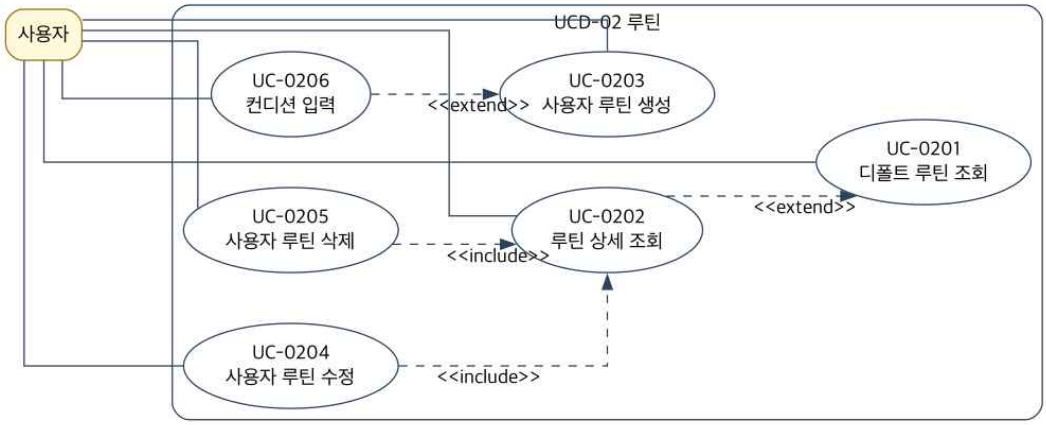
		상점 조회	UC-0602
		아이템 구매	UC-0603
경쟁	UCD-07	경쟁 참가	UC-0701
		경쟁 상대 조회	UC-0702
		경쟁 기록 조회	UC-0703
		경쟁 결과 확인	UC-0704
		경쟁 보상 지급	UC-0705

2.2 Use Case Diagram

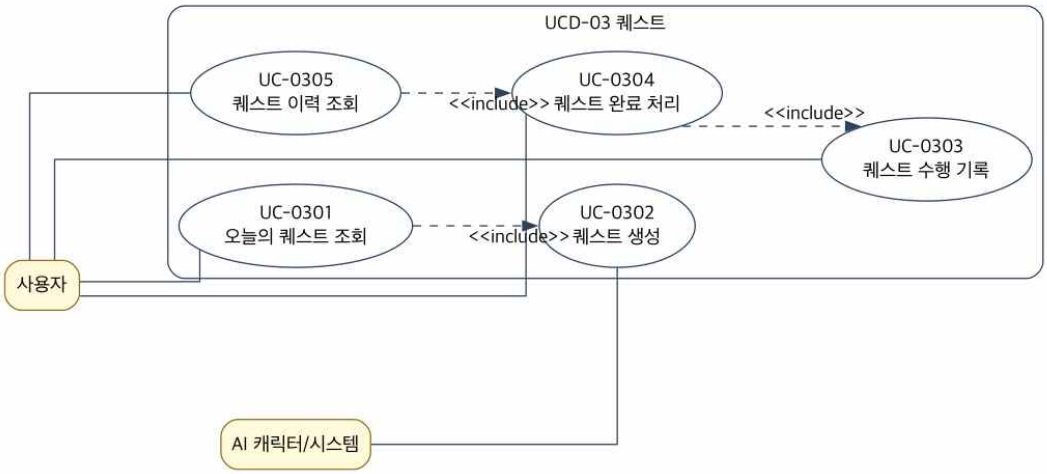
유즈케이스 다이어그램 식별자	UCD-01	유즈케이스 다이어그램 명	사용자



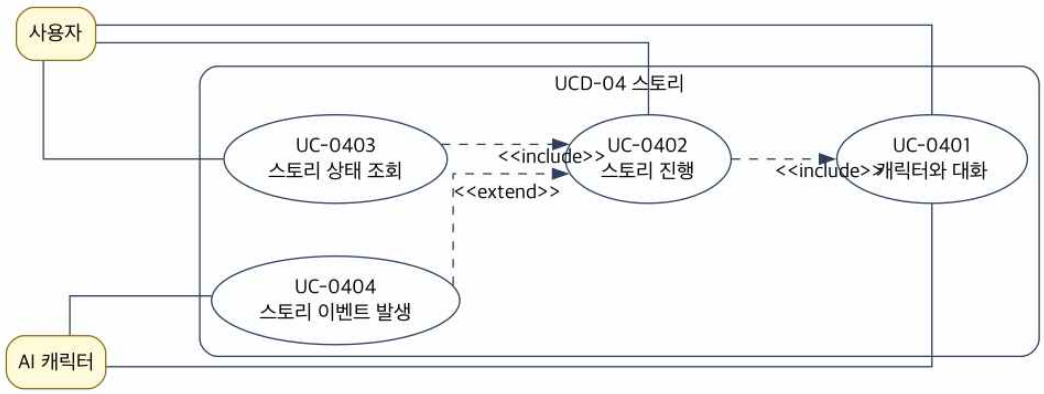
유즈케이스 다이어그램 식별자	UCD-02	유즈케이스 다이어그램 명	루틴



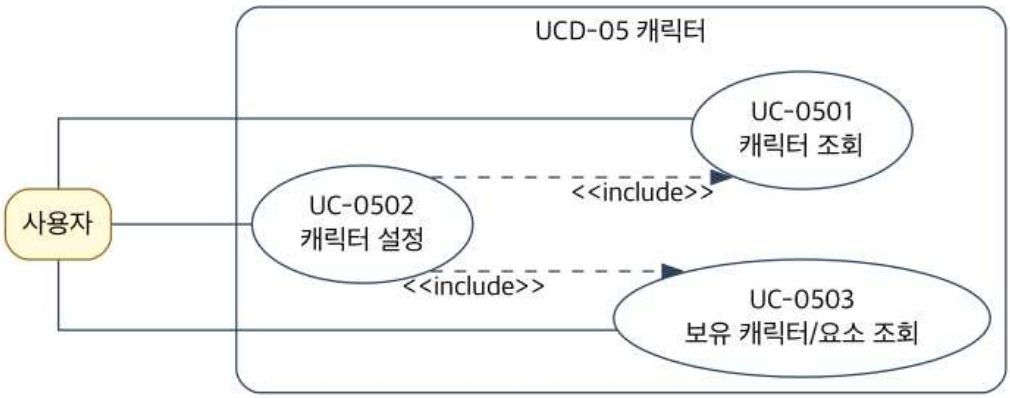
유즈케이스 다이어그램 식별자	UCD-03	유즈케이스 다이어그램 명	퀘스트



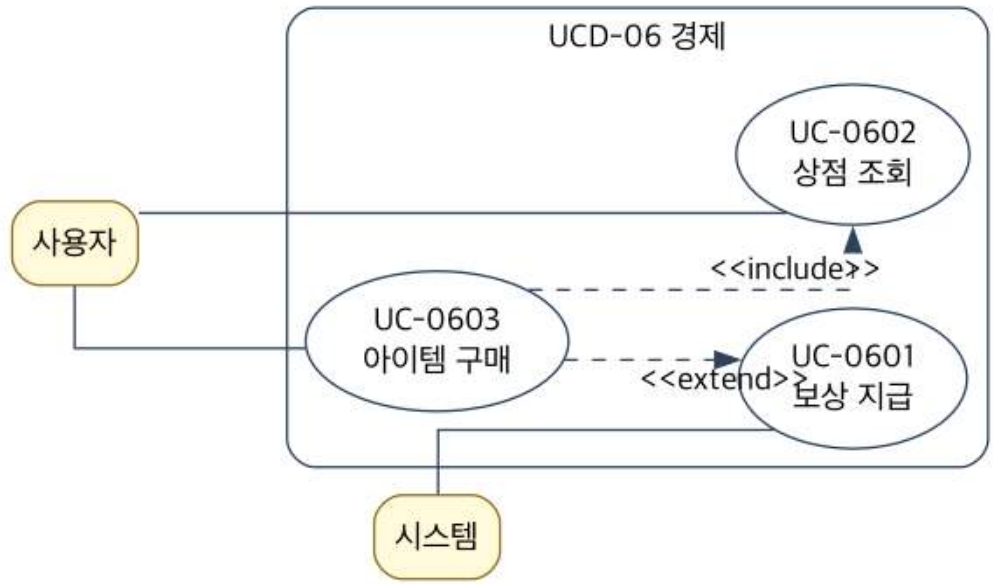
유즈케이스 다이어그램 식별자	UCD-04	유즈케이스 다이어그램 명	스토리



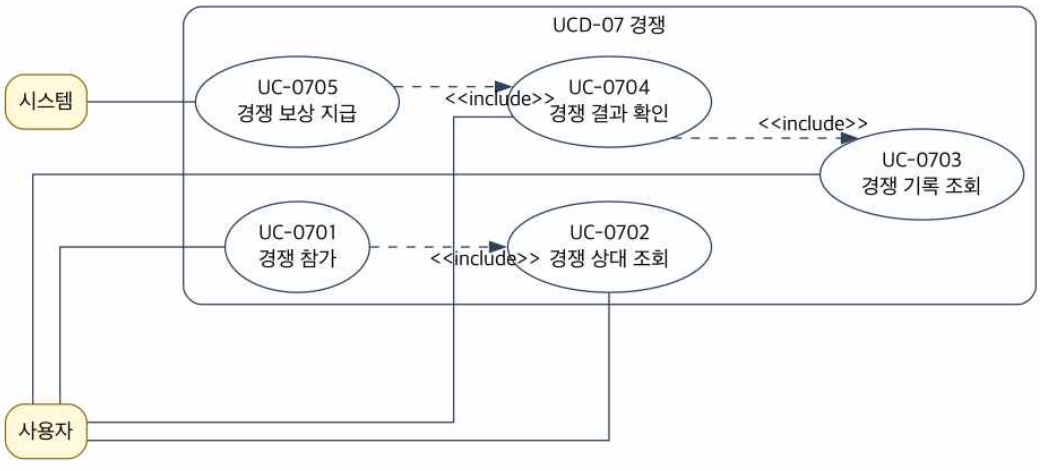
유즈케이스 다이어그램 식별자	UCD-05	유즈케이스 다이어그램 명	캐릭터



유즈케이스 다이어그램 식별자	UCD-06	유즈케이스 다이어그램 명	경제



유즈케이스 다이어그램 식별자	UCD-07	유즈케이스 다이어그램 명	경쟁



2.3 Use Case Descriptions(유즈케이스 명세)

유즈케이스 식별자	UC-0101	유즈케이스 명	회원가입
액터	비회원 사용자		
목표	서비스 이용을 위한 신규 계정을 생성한다.		
범위	사용자 계정 생성 및 초기 프로필 등록		
선행 조건			
사용자는 회원가입 화면에 접근할 수 있어야 하며 중복되지 않은 이메일 또는 아이디를 보유해야 한다.			
사후 조건			
새 계정이 생성되고 기본 프로필이 저장된다. 이메일 인증 대기 상태가 된다.			
시나리오			
1. 사용자가 회원가입 화면으로 이동한다. 2. 시스템이 가입 입력 폼을 표시한다. 3. 사용자가 필수 정보를 입력하고 약관에 동의한다. 4. 시스템이 입력값과 중복 여부를 검증한다. 5. 검증이 완료되면 계정을 생성하고 완료 메시지를 표시한다.			
확장			
이메일 중복, 비밀번호 형식 오류, 필수값 누락, 약관 미동의 시 가입을 중단하고 오류 메시지를 표시한다.			

유즈케이스 식별자	UC-0102	유즈케이스 명	로그인
액터	등록 회원		
목표	사용자를 인증하고 JWT 토큰을 발급하여 서비스에 접속시킨다.		
범위	이메일 또는 아이디와 비밀번호 기반 JWT 인증		
선행 조건			
사용자는 가입된 계정을 보유해야 하며 계정이 탈퇴 또는 잠금 상태가 아니어야 한다.			
사후 조건			
인증이 성공하면 액세스 토큰과 리프레시 토큰이 발급되고 메인 화면으로 이동한다.			
시나리오			
1. 사용자가 로그인 화면으로 이동한다. 2. 시스템이 로그인 폼을 표시한다. 3. 사용자가 계정 정보를 입력하고 로그인 요청을 보낸다. 4. 시스템이 계정 존재 여부와 비밀번호를 검증한다. 5. 인증이 성공하면 시스템이 액세스 토큰과 리프레시 토큰을 발급하고 메인 화면을 제공한다.			
확장			
비밀번호 불일치, 존재하지 않는 계정, 휴면 또는 잠금 계정이면 토큰 발급 없이 로그인에 실패하고 안내 메시지를 표시한다.			

유즈케이스 식별자	UC-0103	유즈케이스 명	로그아웃
액터	로그인 사용자		
목표	JWT 기반 인증을 종료하고 안전하게 서비스에서 이탈한다.		
범위	클라이언트 토큰 삭제 및 서버 측 리프레시 토큰 무효화		
선행 조건			
사용자가 유효한 인증 토큰으로 로그인된 상태여야 한다.			
사후 조건			
저장된 액세스 토큰과 리프레시 토큰이 제거되고, 서버에 저장된 리프레시 토큰은 무효화된다. 사용자는 로그인 전 화면으로 이동한다.			
시나리오			
1. 사용자가 로그아웃 메뉴를 선택한다. 2. 시스템이 로그아웃 요청의 인증 상태를 확인한다. 3. 시스템이 서버에 저장된 리프레시 토큰을 무효화한다. 4. 클라이언트가 저장된 액세스 토큰과 리프레시 토큰을 삭제한다. 5. 시스템이 로그인 화면 또는 초기 화면으로 이동시킨다.			
확장			
리프레시 토큰이 이미 만료되었거나 존재하지 않더라도 클라이언트의 토큰 정보를 제거하고 초기 화면으로 이동시킨다.			

유즈케이스 식별자	UC-0104	유즈케이스 명	아이디 찾기
액터	비회원 사용자		
목표	사용자가 본인 계정의 아이디를 확인할 수 있도록 한다.		
범위	본인 확인 기반 아이디 조회		
선행 조건			
사용자는 계정 생성 시 등록한 이메일 또는 휴대전화 정보를 알고 있어야 한다.			
사후 조건			
본인 확인이 완료되면 아이디가 사용자에게 안내된다.			
시나리오			
1. 사용자가 아이디 찾기 화면으로 이동한다. 2. 시스템이 본인 확인 정보를 요청한다. 3. 사용자가 이메일 또는 휴대전화 정보를 입력한다. 4. 시스템이 일치 여부를 검증한다. 5. 검증 성공 시 등록된 아이디를 안내한다.			
확장			
입력 정보가 일치하지 않으면 조회를 중단하고 재입력을 요청한다.			

유즈케이스 식별자	UC-0105	유즈케이스 명	비밀번호 재설정
액터	비회원 사용자 또는 로그인 불가 사용자		
목표	본인 확인 후 새로운 비밀번호로 계정 접근 권한을 회복한다.		
범위	비밀번호 초기화 및 재설정		
선행 조건			
사용자는 본인 확인이 가능한 이메일 또는 휴대전화 정보를 보유해야 한다.			
사후 조건			
비밀번호가 새 값으로 변경되고 이후 새 비밀번호로 로그인할 수 있다.			
시나리오			
1. 사용자가 비밀번호 재설정 화면으로 이동한다. 2. 시스템이 본인 확인 절차를 안내한다. 3. 사용자가 인증 코드를 확인하고 새로운 비밀번호를 입력한다. 4. 시스템이 인증 코드와 비밀번호 형식을 검증한다. 5. 시스템이 비밀번호를 변경하고 완료 메시지를 표시한다.			
확장			
인증 코드 만료, 비밀번호 형식 오류, 본인 확인 실패 시 재설정을 중단하고 다시 시도하도록 안내한다.			

유즈케이스 식별자	UC-0106	유즈케이스 명	마이페이지 조회
액터	로그인 사용자		
목표	사용자의 계정 및 활동 정보를 한 곳에서 확인한다.		
범위	프로필, 운동 이력, 설정 요약 조회		
선행 조건			
사용자가 로그인된 상태여야 한다.			
사후 조건			
사용자는 본인 프로필, 운동 관련 설정 및 주요 활동 정보를 확인한다.			
시나리오			
1. 사용자가 마이페이지 메뉴를 선택한다. 2. 시스템이 사용자 인증 상태를 확인한다. 3. 시스템이 프로필, 루틴, 활동 요약 정보를 조회한다. 4. 시스템이 마이페이지 화면에 정보를 표시한다.			
확장			
필요한 정보 일부를 불러오지 못하면 가능한 항목만 먼저 표시하고 오류 영역을 안내한다.			

유즈케이스 식별자	UC-0107	유즈케이스 명	사용자 정보 수정
액터	로그인 사용자		
목표	사용자 프로필 및 계정 정보를 최신 상태로 유지한다.		
범위	사용자 개인정보 수정		
선행 조건			
사용자가 로그인되어 있고 마이페이지 또는 설정 화면에 접근할 수 있어야 한다.			
사후 조건			
수정 가능한 사용자 정보가 저장되고 이후 화면에 반영된다.			
시나리오			
1. 사용자가 사용자 정보 수정 화면으로 이동한다. 2. 시스템이 현재 저장된 사용자 정보를 표시한다. 3. 사용자가 수정할 항목을 입력한다. 4. 시스템이 형식과 중복 여부를 검증한다. 5. 검증이 완료되면 변경 내용을 저장한다.			
확장			
입력 형식 오류, 중복 닉네임, 저장 실패 시 수정 결과를 반영하지 않고 오류 메시지를 제공한다.			

유즈케이스 식별자	UC-0108	유즈케이스 명	프로필 설정
액터	로그인 사용자		
목표	사용자의 공개 프로필 정보를 구성한다.		
범위	닉네임, 프로필 이미지 등 프로필 설정		
선행 조건			
사용자가 로그인되어 있고 프로필 수정 권한을 보유해야 한다.			
사후 조건			
프로필 정보가 저장되어 다른 화면에서 표시된다.			
시나리오			
1. 사용자가 프로필 설정 화면을 연다. 2. 시스템이 현재 프로필 정보를 표시한다. 3. 사용자가 프로필 이미지나 닉네임을 변경한다. 4. 시스템이 입력값을 검증한다. 5. 시스템이 프로필 변경 내용을 저장하고 결과를 표시한다.			
확장			
설정 실패 시 변경을 취소하고 오류를 안내한다.			

유즈케이스 식별자	UC-0109	유즈케이스 명	운동 경험 설정
액터	로그인 사용자		
목표	사용자의 운동 수준을 반영해 맞춤형 추천에 활용한다.		
범위	운동 경력, 운동 빈도, 숙련도 설정		
선행 조건			
사용자가 로그인되어 있고 설정 화면에 접근할 수 있어야 한다.			
사후 조건			
운동 경험 정보가 저장되어 루틴 및 퀘스트 추천에 반영된다.			
시나리오			
1. 사용자가 운동 경험 설정 화면으로 이동한다. 2. 시스템이 현재 운동 경험 정보를 표시한다. 3. 사용자가 운동 경력과 빈도, 숙련도를 선택한다. 4. 시스템이 선택값을 저장한다. 5. 시스템이 반영 완료 메시지를 표시한다.			
확장			
선택값이 누락되거나 저장 과정에 실패하면 기존 값을 유지하고 재시도를 요청한다.			

유즈케이스 식별자	UC-0110	유즈케이스 명	선호 운동 설정
액터	로그인 사용자		
목표	사용자가 선호하는 운동 종목과 목표를 설정한다.		
범위	선호 운동 종목, 시간대, 운동 목표 설정		
선행 조건			
사용자가 로그인되어 있고 개인화 설정 화면에 접근할 수 있어야 한다.			
사후 조건			
선호 운동 정보가 저장되어 추천 루틴과 퀘스트에 반영된다.			
시나리오			
1. 사용자가 선호 운동 설정 화면을 연다. 2. 시스템이 선택 가능한 운동 종목과 목표 항목을 표시한다. 3. 사용자가 선호 운동과 목표를 선택한다. 4. 시스템이 설정값을 저장한다. 5. 시스템이 개인화 반영 완료 메시지를 표시한다.			
확장			
선택값 누락, 저장 실패, 지원하지 않는 항목 선택 시 설정을 반영하지 않고 오류를 표시한다.			

유즈케이스 식별자	UC-0111	유즈케이스 명	푸시 알림
액터	로그인 사용자		
목표	루틴, 퀘스트, 경쟁 관련 알림을 설정하고 필요한 시점에 안내받는다.		
범위	푸시 알림 수신 동의, 알림 항목 선택, 알림 발송		
선행 조건			
사용자가 로그인되어 있고 알림 설정 화면에 접근할 수 있어야 하며, 기기에서 알림 수신 권한이 허용되어야 한다.			
사후 조건			
사용자의 푸시 알림 설정이 저장되고, 이후 설정된 이벤트 발생 시 알림이 발송된다.			
시나리오			
1. 사용자가 알림 설정 화면으로 이동한다. 2. 시스템이 현재 푸시 알림 수신 여부와 알림 항목을 표시한다. 3. 사용자가 루틴, 퀘스트, 경쟁 관련 알림 수신 여부를 설정한다. 4. 시스템이 변경된 설정을 저장한다. 5. 시스템이 설정된 조건에 따라 푸시 알림을 발송한다.			
확장			
알림 권한이 거부되었거나 저장에 실패하면 시스템은 알림 설정이 적용되지 않았음을 안내하고 재설정을 요청한다.			

유즈케이스 식별자	UC-0201	유즈케이스 명	디폴트 루틴 조회
액터	로그인 사용자		
목표	서비스가 제공하는 기본 운동 루틴 목록을 확인한다.		
범위	기본 루틴 목록 및 요약 정보 조회		
선행 조건			
사용자가 로그인되어 있어야 하며 루틴 메뉴에 접근할 수 있어야 한다.			
사후 조건			
사용자는 선택 가능한 기본 루틴 목록과 각 루틴의 핵심 정보를 확인한다.			
시나리오			
1. 사용자가 루틴 메뉴로 이동한다. 2. 시스템이 기본 루틴 목록을 조회한다. 3. 시스템이 루틴 이름, 난이도, 목표 정보를 표시한다. 4. 사용자가 원하는 루틴을 탐색한다.			
확장			
기본 루틴 데이터가 없거나 조회에 실패하면 빈 목록 또는 오류 메시지를 표시한다.			

유즈케이스 식별자	UC-0202	유즈케이스 명	루틴 상세 조회
액터	로그인 사용자		
목표	선택한 루틴의 세부 운동 구성을 확인한다.		
범위	루틴 상세 항목, 운동 순서, 소요 시간 조회		
선행 조건			
사용자가 기본 루틴 또는 사용자 루틴 목록을 조회한 상태여야 한다.			
사후 조건			
사용자는 루틴의 상세 구성과 수행 정보를 확인한다.			
시나리오			
1. 사용자가 특정 루틴을 선택한다. 2. 시스템이 해당 루틴의 상세 정보를 조회한다. 3. 시스템이 운동 목록, 반복 횟수, 시간 정보를 표시한다. 4. 사용자가 루틴 수행 여부를 판단한다.			
확장			
루틴 상세 데이터가 존재하지 않으면 요약 정보만 보여주고 상세 조회 불가를 안내한다.			

유즈케이스 식별자	UC-0203	유즈케이스 명	사용자 루틴 생성
액터	로그인 사용자		
목표	사용자에게 맞는 개인 운동 루틴을 새로 만든다.		
범위	사용자 맞춤 루틴 생성 및 저장		
선행 조건			
사용자가 로그인되어 있어야 하며 루틴 생성 화면에 접근 가능해야 한다.			
사후 조건			
새 사용자 루틴이 저장되고 이후 목록에서 조회 가능하다.			
시나리오			
1. 사용자가 루틴 생성 화면을 연다. 2. 시스템이 루틴 이름과 운동 항목 입력 폼을 제공한다. 3. 사용자가 운동 종목, 순서, 반복 횟수, 시간을 입력한다. 4. 시스템이 입력값을 검증한다. 5. 시스템이 새 루틴을 저장하고 생성 완료를 알린다.			
확장			
필수 항목 누락, 중복 루틴명, 저장 실패 시 생성 처리를 중단하고 재입력을 요청한다.			

유즈케이스 식별자	UC-0204	유즈케이스 명	사용자 루틴 수정
액터	로그인 사용자		
목표	기존 사용자 루틴을 현재 상태에 맞게 변경한다.		
범위	사용자 생성 루틴 편집 및 재저장		
선행 조건			
사용자가 수정 대상 루틴의 소유자여야 하며 해당 루틴을 조회한 상태여야 한다.			
사후 조건			
변경된 루틴 정보가 저장되고 이후 조회 시 수정 내용이 반영된다.			
시나리오			
1. 사용자가 수정할 루틴을 선택한다. 2. 시스템이 기존 루틴 정보를 편집 가능한 상태로 표시한다. 3. 사용자가 운동 구성이나 반복 횟수를 수정한다. 4. 시스템이 변경값을 검증한다. 5. 시스템이 수정된 루틴을 저장하고 완료 메시지를 표시한다.			
확장			
권한이 없거나 검증에 실패하면 수정 내용을 저장하지 않고 오류를 안내한다.			

유즈케이스 식별자	UC-0205	유즈케이스 명	사용자 루틴 삭제
액터	로그인 사용자		
목표	더 이상 사용하지 않는 개인 루틴을 제거한다.		
범위	사용자 생성 루틴 삭제		
선행 조건			
사용자가 삭제 대상 루틴의 소유자여야 하며 해당 루틴을 조회한 상태여야 한다.			
사후 조건			
선택한 루틴이 목록에서 제거되고 이후 조회할 수 없다.			
시나리오			
1. 사용자가 삭제할 루틴을 선택한다. 2. 시스템이 삭제 확인 메시지를 표시한다. 3. 사용자가 삭제를 최종 확인한다. 4. 시스템이 루틴 데이터를 삭제한다. 5. 시스템이 목록을 갱신하고 삭제 완료를 안내한다.			
확장			
삭제 권한이 없거나 삭제 과정에서 오류가 발생하면 삭제를 취소하고 오류 메시지를 표시한다.			

유즈케이스 식별자	UC-0206	유즈케이스 명	컨디션 입력
액터	로그인 사용자		
목표	현재 신체 상태와 컨디션 정보를 기록해 추천에 반영한다.		
범위	피로도, 통증, 에너지 수준 등 컨디션 기록		
선행 조건			
사용자가 로그인되어 있어야 하며 루틴 또는 오늘 운동 화면에서 컨디션 입력 기능에 접근할 수 있어야 한다.			
사후 조건			
입력된 컨디션 정보가 저장되고 퀘스트 생성에 반영된다.			
시나리오			
1. 사용자가 컨디션 입력 화면을 연다. 2. 시스템이 피로도와 상태 항목을 표시한다. 3. 사용자가 현재 상태를 선택하거나 입력한다. 4. 시스템이 값을 저장한다. 5. 시스템이 저장 결과와 반영 예정 내용을 안내한다.			
확장			
입력값이 유효하지 않거나 저장에 실패하면 기존 상태를 유지하고 재입력을 요청한다.			

유즈케이스 식별자	UC-0301	유즈케이스 명	오늘의 퀘스트 조회
액터	로그인 사용자		
목표	오늘 수행해야 할 운동 퀘스트를 확인한다.		
범위	개인 맞춤 일일 퀘스트 목록 조회		
선행 조건			
사용자가 로그인되어 있어야 하며 오늘자 퀘스트가 생성되어 있거나 생성 가능한 상태여야 한다.			
사후 조건			
사용자는 오늘의 퀘스트 목록과 진행 상태를 확인한다.			
시나리오			
1. 사용자가 오늘의 퀘스트 화면으로 이동한다. 2. 시스템이 사용자 상태와 날짜 기준 퀘스트를 조회한다. 3. 시스템이 퀘스트 목록과 보상 정보를 표시한다. 4. 사용자가 수행할 퀘스트를 선택한다.			
확장			
당일 퀘스트가 아직 생성되지 않았다면 시스템이 퀘스트 생성 로직을 호출하거나 빈 상태 화면을 표시한다.			

유즈케이스 식별자	UC-0302	유즈케이스 명	퀘스트 생성
액터	AI 캐릭터/시스템		
목표	진행 중인 게임 스토리와 사용자 루틴에 맞는 퀘스트를 생성한다.		
범위	스토리 퀘스트 자동 생성		
선행 조건			
사용자의 운동 경험, 선호 운동, 현재 루틴 또는 컨디션 정보가 존재해야 한다. 사용자가 진행 중인 게임의 시나리오가 존재해야 한다.			
사후 조건			
사용자에게 할당 가능한 퀘스트가 생성되어 제공된다.			
시나리오			
1. 시스템이 시나리오 상에서 미리 지정해둔 지점에서 퀘스트의 틀을 생성한다. 2. 사용자가 게임을 진행하며 퀘스트 발생 지점에 진입한다. 3. 시스템이 퀘스트 틀과 사용자의 운동 루틴을 이용하여 퀘스트를 생성한다. 3. 시스템이 퀘스트 보상을 계산한다. 4. 시스템이 생성된 퀘스트를 서브 시스템에 제공한다.			
확장			
추천 기준이 부족하거나 생성에 실패하면 기본 퀘스트를 생성하거나 생성 실패 상태를 기록한다.			

유즈케이스 식별자	UC-0303	유즈케이스 명	퀘스트 수행 기록
액터	로그인 사용자		
목표	퀘스트 수행 과정과 결과를 기록한다.		
범위	퀘스트 진행 상태 및 운동 수행 데이터 기록		
선행 조건			
사용자가 퀘스트를 조회했고 수행 가능한 상태여야 한다.			
사후 조건			
퀘스트 진행도와 운동 수행 기록이 저장된다.			
시나리오			
1. 사용자가 수행 중인 퀘스트를 선택한다. 2. 시스템이 기록 가능한 입력 항목을 표시한다. 3. 사용자가 수행 결과나 진행도를 입력한다. 4. 시스템이 입력값을 검증하고 저장한다. 5. 시스템이 갱신된 진행 상태를 표시한다.			
확장			
입력값이 누락되거나 저장이 실패하면 기존 진행 상태를 유지하고 재시도를 요청한다.			

유즈케이스 식별자	UC-0304	유즈케이스 명	퀘스트 완료 처리
액터	로그인 사용자		
목표	수행이 끝난 퀘스트를 완료 상태로 전환한다.		
범위	퀘스트 완료 검증 및 상태 변경		
선행 조건			
해당 퀘스트의 수행 기록이 존재하고 완료 조건을 만족해야 한다.			
사후 조건			
퀘스트 상태가 완료로 변경되고 보상 지급 대상이 된다.			
시나리오			
1. 사용자가 완료 가능한 퀘스트를 선택한다. 2. 시스템이 완료 조건 충족 여부를 검증한다. 3. 시스템이 퀘스트 상태를 완료로 변경한다. 4. 시스템이 보상 지급 이벤트를 생성한다. 5. 시스템이 완료 결과를 사용자에게 안내한다.			
확장			
완료 조건이 충족되지 않으면 완료 처리를 중단하고 부족한 조건을 안내한다.			

유즈케이스 식별자	UC-0305	유즈케이스 명	퀘스트 이력 조회
액터	로그인 사용자		
목표	과거 퀘스트 수행 기록과 완료 이력을 확인한다.		
범위	퀘스트 이력 및 결과 조회		
선행 조건			
사용자가 로그인되어 있어야 하며 퀘스트 기록이 존재해야 한다.			
사후 조건			
사용자는 기간별 퀘스트 수행 내역과 완료 결과를 확인한다.			
시나리오			
1. 사용자가 퀘스트 이력 화면을 선택한다. 2. 시스템이 기간 조건 또는 최신순 기준으로 이력을 조회한다. 3. 시스템이 완료 여부와 보상 결과를 함께 표시한다. 4. 사용자가 원하는 기록을 상세 확인한다.			
확장			
기록이 없으면 빈 이력 화면을 표시하고 퀘스트 수행을 유도한다.			

유즈케이스 식별자	UC-0401	유즈케이스 명	캐릭터와 대화
액터	로그인 사용자		
목표	생성된 캐릭터와 대화를 통해 게임을 진행한다.		
범위	캐릭터와의 대화, 상호작용		
선행 조건			
사용자가 로그인되어 있어야 하며 공략중인 캐릭터가 존재하여야 한다.			
사후 조건			
호감도나 스토리 진행도가 올라간다.			
시나리오			
1. 캐릭터(시스템)가 로그인 사용자에게 대화를 건다. 2. 사용자는 여러 선택지 중에 선택하여 대답을 한다. 3. 캐릭터(시스템)는 선택한 답변에 따라 다른 행동과 답변을 한다.			
확장			
답변 선택이 누락되거나 저장이 실패하면 기존 스토리의 진행 상태를 유지하고 재시도를 요청한다.			

유즈케이스 식별자	UC-0402	유즈케이스 명	스토리 진행
액터	로그인 사용자		
목표	스토리 진행을 위한 텍스트를 생성한다. 캐릭터의 대사를 유도한다		
범위	상황이나 장면, 행동을 해설하는 텍스트 생성, 캐릭터 시스템의 대사 유도		
선행 조건			
사용자가 로그인되어 있어야 하며 진행 중인 스토리가 존재하거나, 신규로 만들어야 한다.			
사후 조건			
스토리 진행도가 올라간다.			
시나리오			
1. 시스템이 스토리의 세계관이나 상세 설정 등을 설명하는 텍스트를 제공한다. 2. 캐릭터 시스템이 상황에 맞는 대사를 생성할 수 있도록 정보를 제공하고 관리한다. 3. 사용자와 캐릭터 시스템의 대화를 관리하고, 다음 상황으로 유도할 수 있는 텍스트를 생성하고 제공한다.			
확장			
텍스트 생성에 실패하면 기존 스토리의 진행 상태를 유지하고 재시도를 요청한다.			

유즈케이스 식별자	UC-0403	유즈케이스 명	스토리 상태 조회
액터	로그인 사용자		
목표	스토리의 진행 상태나, 정보, 스토리 퀘스트 완료 여부를 조회한다.		
범위	스토리의 진행 상태나, 정보, 스토리 퀘스트 완료 여부를 조회		
선행 조건			
사용자가 로그인되어 있어야 하며 진행 중인 스토리가 존재하거나, 신규로 만들어야 한다.			
사후 조건			
사용자는 스토리의 진행 상태, 스토리 퀘스트 완료 여부를 확인한다.			
시나리오			
1. 사용자가 스토리 상태 조회 버튼을 선택한다. 2. 시스템이 스토리의 진행 상태, 스토리 퀘스트 완료 여부를 조회한다. 3. 시스템이 완료 여부와 진행상태를 표시한다. 4. 사용자가 원하는 기록을 상세 확인한다.			
확장			
기록이 없으면 빈 이력 화면을 표시한다.			

유즈케이스 식별자	UC-0404	유즈케이스 명	스토리 이벤트 발생
액터	로그인 사용자		
목표	이후 스토리 진행에 영향을 줄 수 있는 이벤트를 발생시킨다.		
범위	이벤트 발생시키고 사용자 응답에 따른 이후 스토리 생성		
선행 조건			
사용자가 로그인되어 있어야 하며 이벤트 발생 지점까지 스토리가 진행되어야 한다			
사후 조건			
스토리 전개 내용이 사용자의 선택에 따라 변화한다. 스토리 진행도가 올라간다.			
시나리오			
1. 사용자가 게임을 진행하며 이벤트 발생 지점에 진입한다. 2. 시스템이 시나리오 상에서 미리 지정해둔 지점에서 사용자 응답 선택지들을 생성한다. 3. 사용자가 선택지를 고른다. 4. 시스템이 응답으로 받은 선택지에 따라 이후 스토리 전개를 변형하여 제공한다			
확장			
답변 선택이 누락되거나 저장이 실패하면 기존 스토리의 진행 상태를 유지하고 재시도를 요청한다.			

유즈케이스 식별자	UC-0501	유즈케이스 명	캐릭터 조회
액터	로그인 사용자		
목표	현재 선택된 캐릭터와 관련 상태 정보를 확인한다.		
범위	캐릭터 정보 및 적용 상태 조회		
선행 조건			
사용자가 로그인되어 있어야 하며 캐릭터 기능에 접근할 수 있어야 한다.			
사후 조건			
사용자는 현재 선택된 캐릭터와 보유 현황을 확인한다.			
시나리오			
1. 사용자가 캐릭터 화면으로 이동한다. 2. 시스템이 현재 선택된 캐릭터와 보유한 캐릭터 또는 관련 요소 정보를 조회한다. 3. 시스템이 캐릭터 미리보기와 사용 가능 상태를 표시한다. 4. 사용자가 추가 설정 여부를 결정한다.			
확장			
적용된 캐릭터 데이터가 없으면 기본 캐릭터를 표시하고 초기 설정을 유도한다.			

유즈케이스 식별자	UC-0502	유즈케이스 명	캐릭터 설정
액터	로그인 사용자		
목표	보유한 캐릭터 또는 관련 요소를 선택하여 캐릭터 상태를 변경한다.		
범위	캐릭터 선택 및 적용 상태 변경		
선행 조건			
사용자가 변경 가능한 캐릭터 또는 관련 요소를 보유하고 있어야 한다.			
사후 조건			
선택한 캐릭터 또는 관련 요소가 적용되고 변경 내용이 저장된다.			
시나리오			
1. 사용자가 캐릭터 설정 화면을 연다. 2. 시스템이 보유한 캐릭터 또는 관련 요소 목록을 표시한다. 3. 사용자가 적용할 대상을 선택한다. 4. 시스템이 미리보기를 갱신한다. 5. 사용자가 저장을 선택하면 시스템이 변경 내용을 반영한다.			
확장			
보유하지 않은 캐릭터 또는 관련 요소는 적용할 수 없으며 저장 실패 시 이전 설정으로 복원한다.			

유즈케이스 식별자	UC-0503	유즈케이스 명	보유 캐릭터/요소 조회
액터	로그인 사용자		
목표	현재 보유한 캐릭터와 캐릭터 관련 요소를 확인한다.		
범위	캐릭터 보유 현황 조회		
선행 조건			
사용자가 로그인되어 있어야 한다.			
사후 조건			
사용자는 보유 캐릭터 또는 관련 요소의 사용 가능 여부를 확인한다.			
시나리오			
1. 사용자가 캐릭터 보유 현황 화면을 연다. 2. 시스템이 보유 캐릭터 및 관련 요소 데이터를 조회한다. 3. 시스템이 목록과 상태를 표시한다. 4. 사용자가 상세 정보를 확인한다.			
확장			
보유한 캐릭터나 관련 요소가 없으면 빈 목록 화면을 표시한다.			

유즈케이스 식별자	UC-0601	유즈케이스 명	보상 지급
액터	시스템		
목표	퀘스트나 경쟁 결과에 따라 적절한 보상을 지급한다.		
범위	재화, 경험치, 아이템 보상 지급		
선행 조건			
보상 지급의 트리거가 되는 완료 이벤트가 발생해야 한다.			
사후 조건			
사용자의 재화, 경험치 또는 아이템 보유량이 갱신된다.			
시나리오			
1. 시스템이 보상 지급 이벤트를 감지한다. 2. 시스템이 보상 규칙과 지급 대상을 조회한다. 3. 시스템이 지급할 재화나 아이템을 계산한다. 4. 시스템이 사용자 계정에 보상을 반영한다. 5. 시스템이 지급 결과를 기록하고 사용자에게 알린다.			
확장			
지급 규칙 오류나 저장 실패가 발생하면 보상 지급을 보류하고 재처리 대상으로 기록한다.			

유즈케이스 식별자	UC-0602	유즈케이스 명	상점 조회
액터	로그인 사용자		
목표	구매 가능한 아이템과 가격 정보를 확인한다.		
범위	상점 상품 목록 및 가격 조회		
선행 조건			
사용자가 로그인되어 있고 상점 기능에 접근할 수 있어야 한다.			
사후 조건			
사용자는 구매 가능한 상품과 가격, 보유 재화를 확인한다.			
시나리오			
1. 사용자가 상점 화면으로 이동한다. 2. 시스템이 판매 중인 상품 목록을 조회한다. 3. 시스템이 상품 이미지, 가격, 구매 조건을 표시한다. 4. 사용자가 구매할 상품을 선택한다.			
확장			
상점 데이터가 없거나 조회에 실패하면 판매 상품이 없음을 안내한다.			

유즈케이스 식별자	UC-0603	유즈케이스 명	아이템 구매
액터	로그인 사용자		
목표	보유 재화를 사용해 원하는 아이템을 구매한다.		
범위	재화 차감 및 아이템 지급		
선행 조건			
사용자가 충분한 재화를 보유하고 구매 가능한 상품을 선택해야 한다.			
사후 조건			
구매한 아이템이 인벤토리에 추가되고 재화가 차감된다.			
시나리오			
1. 사용자가 구매할 상품을 선택한다. 2. 시스템이 가격과 보유 재화를 비교한다. 3. 사용자가 구매를 확정한다. 4. 시스템이 재화를 차감하고 아이템을 지급한다. 5. 시스템이 구매 완료 메시지와 갱신된 보유 현황을 표시한다.			
확장			
재화 부족, 상품 판매 중단, 지급 실패 시 구매를 완료하지 않고 원인을 안내한다.			

유즈케이스 식별자	UC-0701	유즈케이스 명	경쟁 참가
액터	로그인 사용자		
목표	사용자가 경쟁 콘텐츠에 참여해 다른 사용자와 기록을 비교한다.		
범위	경쟁 매칭 및 참가 등록		
선행 조건			
사용자가 로그인되어 있어야 하며 경쟁 참여 조건을 충족해야 한다.			
사후 조건			
사용자가 경쟁 대상 목록 또는 매칭 상태에 등록된다.			
시나리오			
1. 사용자가 경쟁 메뉴를 선택한다. 2. 시스템이 참여 자격과 현재 경쟁 가능 여부를 확인한다. 3. 사용자가 경쟁 참가를 요청한다. 4. 시스템이 참가 등록 또는 매칭 대기 상태로 처리한다. 5. 시스템이 참가 결과를 안내한다.			
확장			
참여 자격을 충족하지 못하면 경쟁 참가를 제한하고 필요한 조건을 안내한다.			

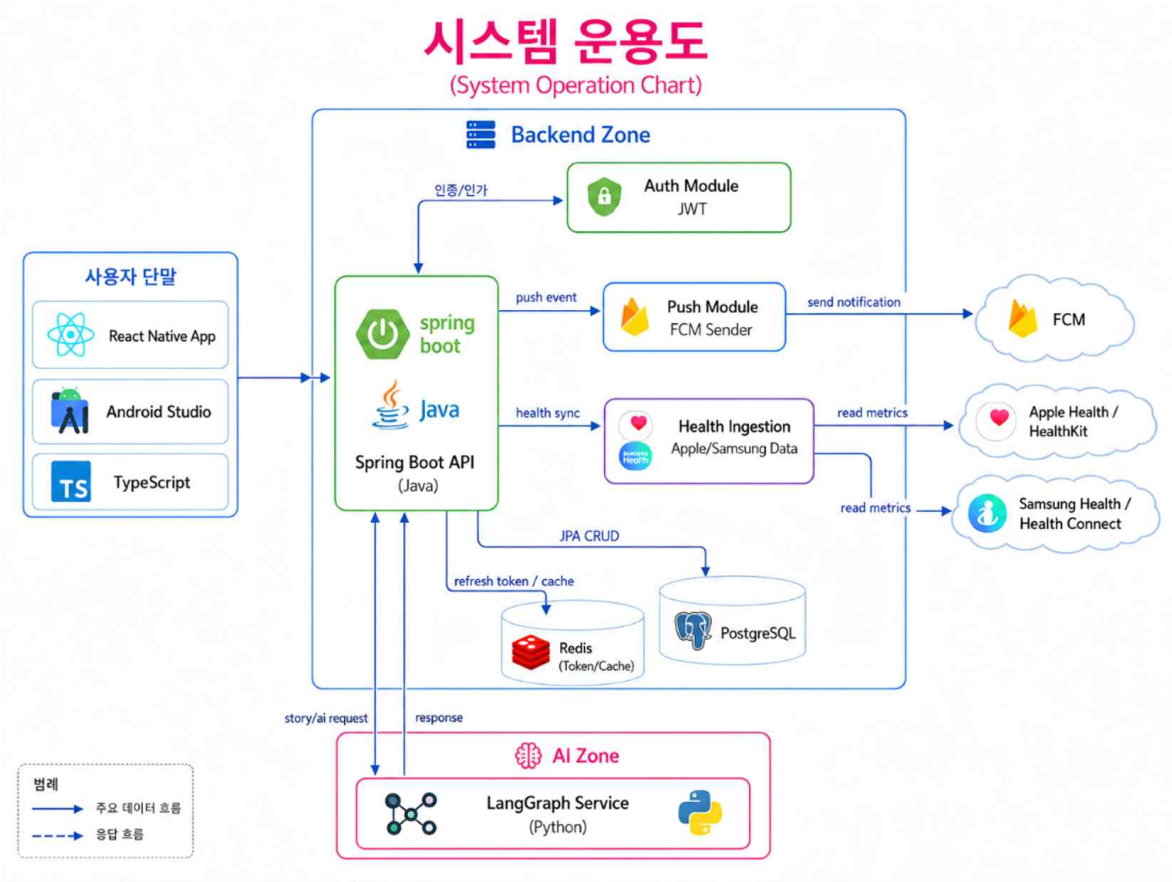
유즈케이스 식별자	UC-0702	유즈케이스 명	경쟁 상대 조회
액터	로그인 사용자		
목표	매칭된 경쟁 상대 정보를 확인한다.		
범위	상대 사용자 및 경쟁 조건 조회		
선행 조건			
사용자가 경쟁에 참가한 상태여야 한다.			
사후 조건			
사용자는 상대 정보, 비교 기준, 경쟁 기간을 확인한다.			
시나리오			
1. 사용자가 경쟁 상대 확인 화면을 연다. 2. 시스템이 매칭 결과를 조회한다. 3. 시스템이 상대 정보와 경쟁 규칙을 표시한다. 4. 사용자가 경쟁 진행 여부를 판단한다.			
확장			
아직 상대가 매칭되지 않았다면 대기 상태를 표시한다.			

유즈케이스 식별자	UC-0703	유즈케이스 명	경쟁 기록 조회
액터	로그인 사용자		
목표	경쟁 중 또는 완료된 경쟁 기록을 확인한다.		
범위	경쟁 진행 기록 및 결과 데이터 조회		
선행 조건			
사용자가 경쟁에 참여한 이력이 있어야 한다.			
사후 조건			
사용자는 경쟁별 수행 기록과 비교 현황을 확인한다.			
시나리오			
1. 사용자가 경쟁 기록 화면을 선택한다. 2. 시스템이 경쟁 참여 이력을 조회한다. 3. 시스템이 기간, 상대, 결과 기준으로 기록을 표시한다. 4. 사용자가 상세 기록을 확인한다.			
확장			
경쟁 기록이 없으면 빈 상태를 표시하고 경쟁 참가를 유도한다.			

유즈케이스 식별자	UC-0704	유즈케이스 명	경쟁 결과 확인
액터	로그인 사용자		
목표	경쟁 종료 후 승패 및 결과를 확인한다.		
범위	경쟁 결과 집계 및 표시		
선행 조건			
경쟁이 종료되었고 결과 산출이 완료되어야 한다.			
사후 조건			
사용자는 승패, 순위, 획득 가능 보상을 확인한다.			
시나리오			
1. 사용자가 경쟁 결과 화면을 연다. 2. 시스템이 종료된 경쟁의 결과를 집계한다. 3. 시스템이 승패, 점수, 순위를 표시한다. 4. 사용자가 결과를 확인한다.			
확장			
집계가 완료되지 않았으면 결과 대기 상태를 표시한다.			

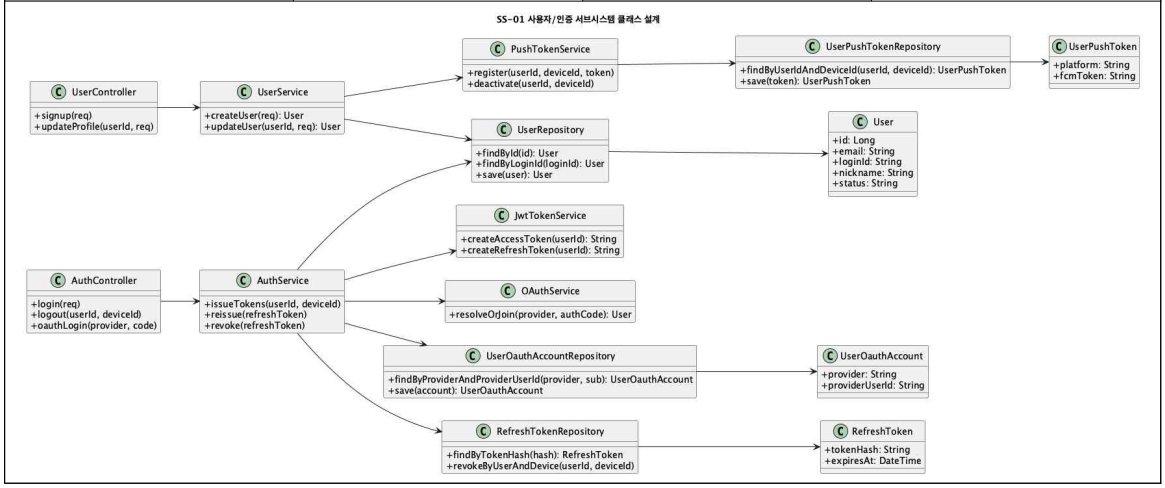
유즈케이스 식별자	UC-0705	유즈케이스 명	경쟁 보상 지급
액터	시스템		
목표	경쟁 결과에 따라 사용자에게 보상을 지급한다.		
범위	경쟁 보상 계산 및 지급		
선행 조건			
경쟁 결과가 확정되어 있고 보상 규칙이 정의되어 있어야 한다.			
사후 조건			
결과에 따른 재화 또는 아이템 보상이 사용자 계정에 반영된다.			
시나리오			
1. 시스템이 경쟁 종료 이벤트를 감지한다. 2. 시스템이 순위와 보상 규칙을 조회한다. 3. 시스템이 사용자별 보상 수량을 계산한다. 4. 시스템이 보상을 지급한다. 5. 시스템이 지급 결과를 기록하고 사용자에게 알린다.			
확장			
보상 계산 오류나 지급 실패가 발생하면 재처리 대상으로 기록하고 임시 안내 메시지를 제공한다.			

3. 시스템 구조도

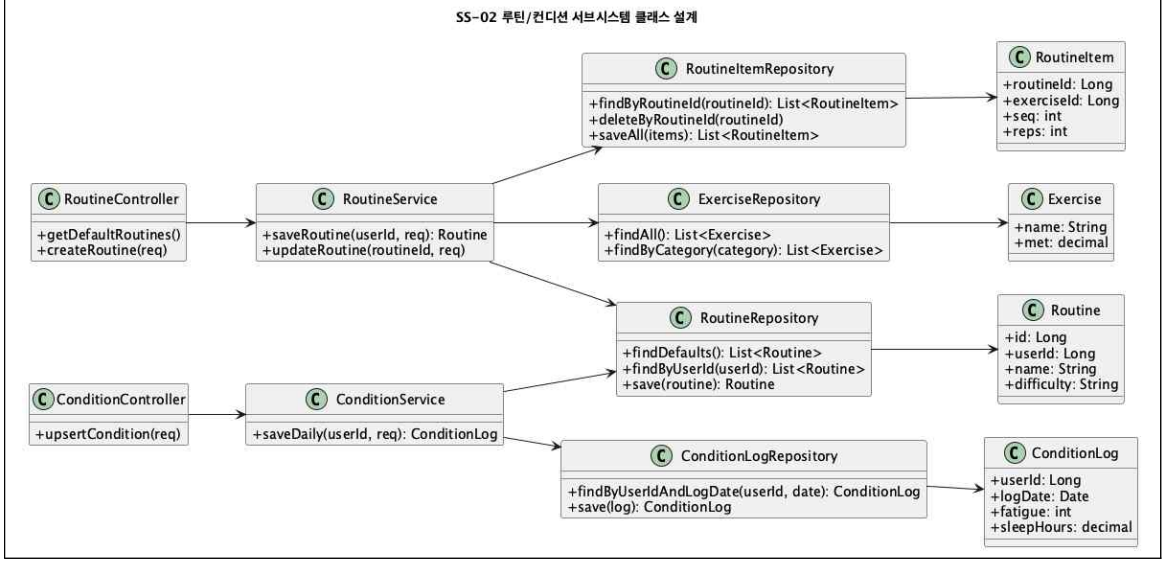


4. 시스템 설계도

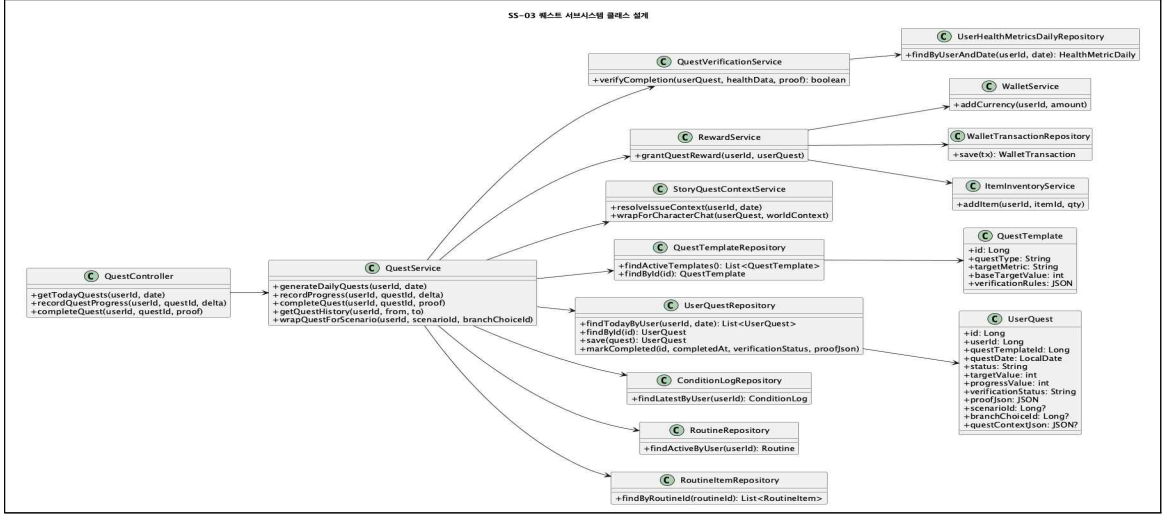
서브 시스템 식별자	SS-01	서브 시스템 명	사용자/인증



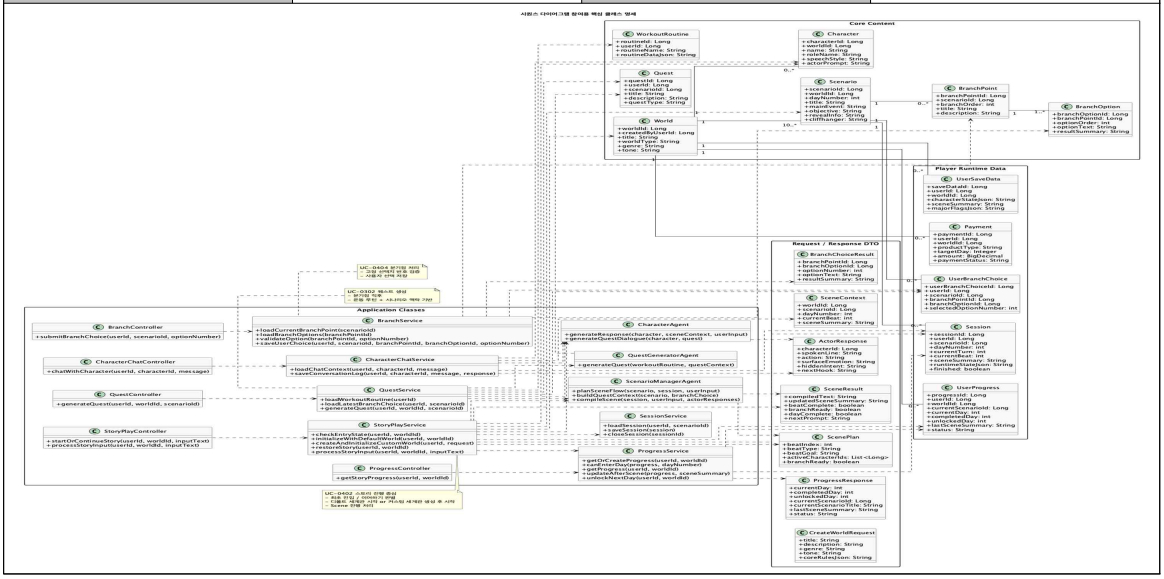
서브 시스템 식별자	SS-02	서브 시스템 명	루틴/컨디션



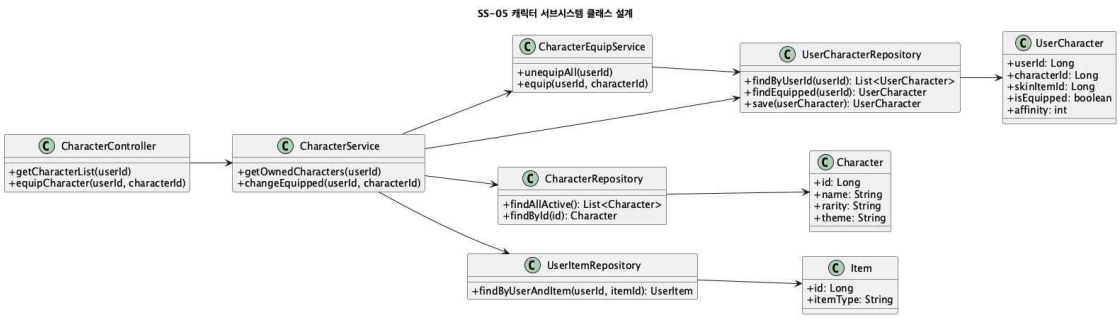
서브 시스템 식별자	SS-03	서브 시스템 명	퀘스트



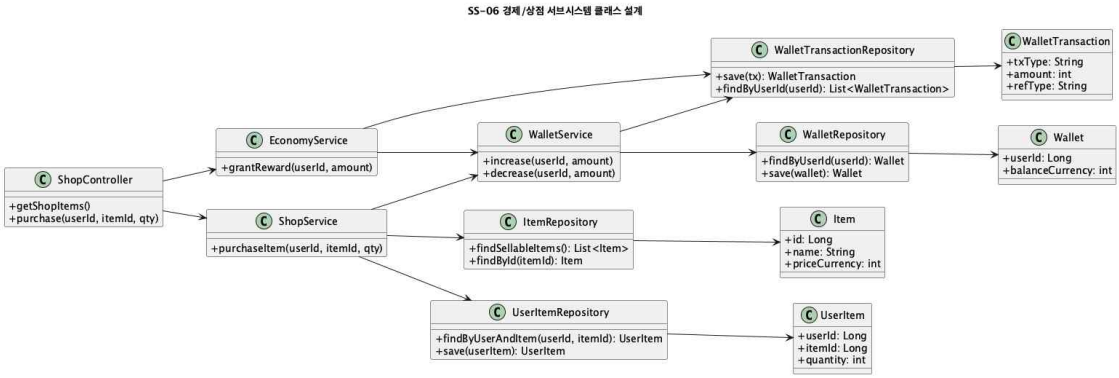
서브 시스템 식별자	SS-04	서브 시스템 명	스토리



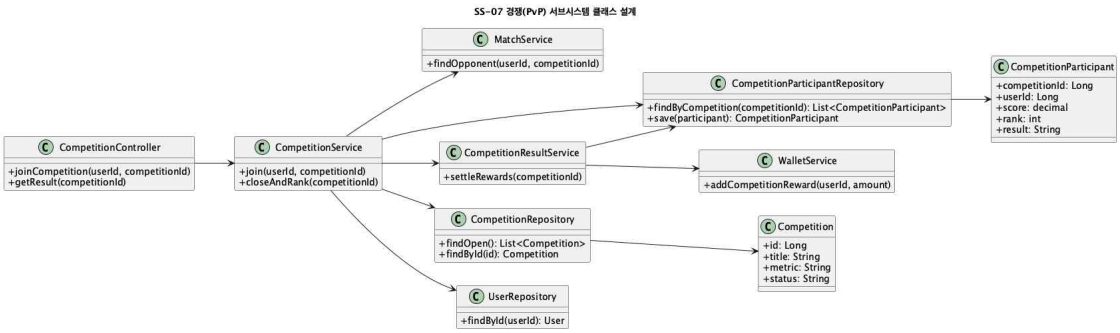
서브 시스템 식별자	SS-05	서브 시스템 명	캐릭터



서브 시스템 식별자	SS-06	서브 시스템 명	경제/상점



서브 시스템 식별자	SS-07	서브 시스템 명	경쟁(PvP)



5. H/W 또는 시스템 구성

식별자	HW-01	구분	서비스 서버
상세 정보	운영 서버: Ubuntu 24.04.3 LTS CPU: Intel N150, 4 Core / 4 Thread Memory: 16GiB, Disk: 512GiB		

6. 구현 기술

본 시스템은 React Native 기반 모바일 앱, Spring Boot 기반 백엔드 API, FastAPI 기반 AI 서버로 구성된다. 모바일은 사용자 화면과 건강 데이터 수집을 담당하고, 백엔드는 인증/루틴/퀘스트/상점/배틀/보상 도메인의 핵심 로직과 데이터 저장을 처리하며, AI 서버는 캐릭터 스토리 생성과 스토리 퀘스트 진행 기능을 제공한다.

6.1 기술 스택

기술	용도
Mobile - React Native 0.85.2 / React 19	Android/iOS 공통 모바일 앱 화면과 사용자 인터랙션 구현
Mobile - TypeScript	모바일 서비스, 화면, API 응답 타입을 정적으로 관리
Mobile - React Navigation	인증, 홈, 루틴, 퀘스트, 상점, 배틀, 스토리 화면 전환 구성
Mobile - Axios	백엔드 REST API 호출 및 도메인별 서비스 모듈 구현
Mobile - Health Connect	걸음 수, 거리, 칼로리, 운동 세션 등 건강 데이터 수집 및 검증
Mobile - Firebase Messaging / Notifee	FCM 토큰 등록, 푸시 알림 수신, 로컬 알림 표시
Backend - Java 21 / Spring Boot 4.0.5	서비스 핵심 REST API와 비즈니스 로직 구현
Backend - Spring MVC / Validation / Security	컨트롤러, 요청값 검증, 인증/인가 필터 구성
Backend - Spring Data JPA / PostgreSQL	사용자, 루틴, 퀘스트, 상점, 배틀, 건강 데이터 영속화
Backend - Redis / JWT	Access Token 검증, Refresh Token 저장, 로그아웃 토큰 블랙리스트 처리
Backend - Firebase Admin / Spring Mail	푸시 알림 발송과 계정 찾기/임시 비밀번호 이메일 발송
Backend - Springdoc OpenAPI	Swagger 기반 API 명세 문서화 및 테스트 지원
AI - Python 3.12 / FastAPI / Uvicorn	스토리 생성 및 플레이 API를 제공하는 AI 서버 구현
AI - LangChain / LangGraph / OpenAI API	캐릭터 기반 대화, 선택지, 스토리 진행, 퀘스트 생성 로직 구현
AI - SQLAlchemy / psycopg / PostgreSQL	시나리오, 캐릭터, 진행 상태, 플레이 로그, 스토리 퀘스트 저장
Infra - Docker / Docker Compose	모바일, 백엔드, AI 서버의 컨테이너 실행 환경 구성

6.2 구현 기술

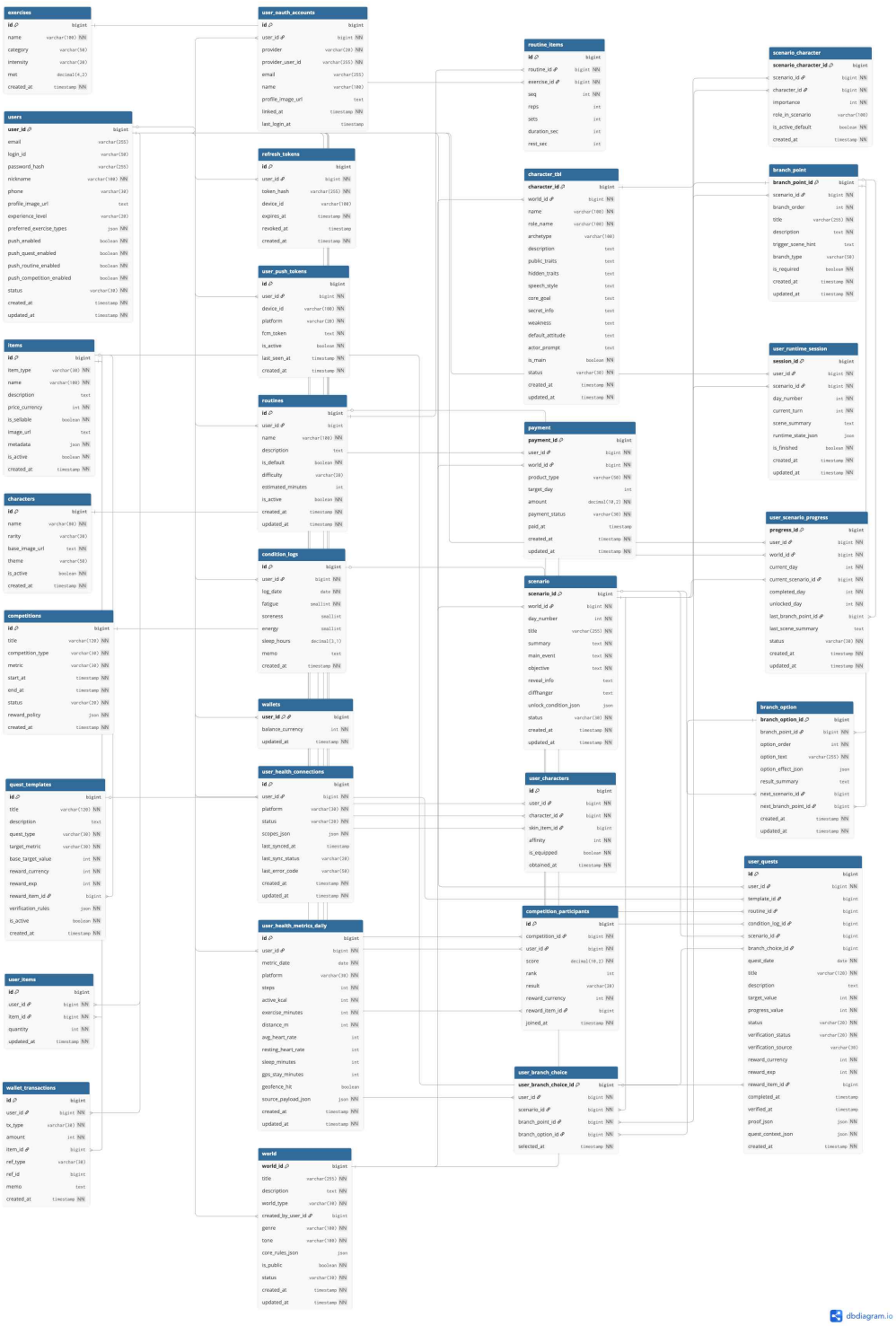
기능명	상세 기능
모바일 화면 및 상태 관리	React Native 컴포넌트와 TypeScript 타입을 기반으로 홈, 루틴, 퀘스트, 상점, 배틀, 스토리 화면을 구성하였다.
모바일 API 연동 계층	AuthService, RoutineService, QuestService, ShopService, BattleService 등 도메인별 Axios 서비스로 REST API 호출을 분리하였다.
건강 데이터 동기화	HealthConnectService에서 Health Connect 권한 요청, 기록 조회, 샘플 변환, 서버 동기화를 수행하고 퀘스트/배틀 검증에 활용한다.
푸시 알림 처리	Firebase Messaging으로 FCM 토큰을 등록하고 Notifee를 이용해 알림 채널, 수신, 클릭 이벤트를 처리한다.
인증 및 보안 처리	Spring Security와 JWT를 사용해 로그인 후 토큰을 발급하고 Refresh Token 저장, Access Token 블랙리스트, 로그아웃 처리를 구현하였다.
사용자/루틴/컨디션 관리	온보딩 프로필 기반 루틴 추천, 커스텀 루틴 생성, 오늘 컨디션 점수와 운동 배율 계산 기능을 구현하였다.
퀘스트 및 보상 처리	오늘 퀘스트 발급, 건강 데이터 기반 완료 검증, 경험치/골드 보상, 배지 지급 로직을 백엔드 서비스로 구현하였다.
상점 및 인벤토리 처리	아이템 조회, 구매, 장착, 사용 기능과 지갑 차감, 거래 내역, 보유 수량 증가 처리를 트랜잭션으로 처리한다.
배틀 매칭 및 결과 산정	대기열 기반 일간/주간 배틀 매칭, 참가자 생성, 건강 데이터와 퀘스트 수행량 기반 점수 산정, 승패/보상 확정을 구현하였다.
건강 데이터 표준화	Health Connect와 HealthKit 원본 record type을 공통 HealthMetricType으로 정규화하고 일별 요약 테이블에 병합 저장한다.
AI 스토리 생성	FastAPI 서버에서 OpenAI/LangChain 기반으로 캐릭터, 장르, 선택지, 장면 이미지를 포함한 스토리 생성 API를 제공한다.
AI 플레이/퀘스트 연동	스토리 진행 로그와 선택지를 저장하고, 스토리 퀘스트 조회/상세/완료 상태를 백엔드 프록시와 연동한다.
백엔드-AI 프록시	AiProxyService와 AiStoryProxyController가 인증 사용자 정보를 포함해 AI 서버 요청을 전달하고 오류 응답을 공통 예외로 변환한다.
API 문서화 및 운영	Springdoc Swagger, 공통 ApiResponse, Dockerfile,

준비	docker-compose를 통해 개발/시연 환경에서 일관되게 실행되도록 구성하였다.

7. DB 구조

7.1 데이터베이스 설계

데이터베이스 설계


exercises
user_oauth_accounts
routine_items
scenario_character
users
refresh_tokens
character_tbl
branch_point
user_push_tokens
user_runtime_session
items
routines
payment
user_scenario_progress
characters
condition_logs
scenario
branch_option
competitions
wallets
user_characters
quest_templates
user_health_connections
user_quests
user_health_metrics_daily
competition_participants
user_items
user_branch_choice
wallet_transactions
world
dbdiagram.io

8. 주요 함수 명세

클래스 명	AuthService
함수 명	signup(SignupRequest request)
함수 설명	회원가입 요청의 중복 여부를 검증하고 비밀번호를 암호화한 뒤 신규 사용자를 생성한다.
인수	
이름	설명
request.loginId	사용자가 입력한 로그인 아이디
request.password	암호화 전 원본 비밀번호
request.nickname	서비스에서 표시할 닉네임
request.email	아이디/비밀번호 찾기에 사용할 이메일
반환 값	
이름	설명
UserProfileResponse	생성된 사용자 프로필, 오늘 컨디션 여부, 가입 보너스 잔액
소스 코드	

```
// auth/service/AuthService.java
    @Transactional
    public UserProfileResponse signup(SignupRequest request) {
        if (userRepository.existsByLoginId(request.loginId())) {
            throw new ApiException(HttpStatus.CONFLICT,
"LOGIN_ID_ALREADY_EXISTS", "Login ID already exists");
        }
        if (userRepository.existsByNickname(request.nickname())) {
            throw new ApiException(HttpStatus.CONFLICT,
"NICKNAME_ALREADY_EXISTS", "Nickname already exists");
        }
        String email = normalize(request.email());
        if (email != null && userRepository.existsByEmail(email)) {
            throw new ApiException(HttpStatus.CONFLICT,
"EMAIL_ALREADY_EXISTS", "이미 등록된 이메일입니다.");
        }

        User user = User.createLocalUser(
                request.loginId(),
                passwordEncoder.encode(request.password()),
                request.nickname(),
                email,
                null
        );

        User savedUser = userRepository.save(user);
        issueSignupBonus(savedUser);
        return UserProfileResponse.from(savedUser, false,
SIGNUP_BONUS_CURRENCY, mediaUrlResolver);
    }
```

클래스 명	AuthService
함수 명	login(LoginRequest request)
함수 설명	로그인 아이디와 비밀번호를 검증하고 기존 리프레시 토큰을 폐기한 뒤 새 토큰 쌍을 발급한다.

인수	
이름	설명
request.loginId	로그인할 사용자 아이디
request.password	사용자가 입력한 비밀번호
반환 값	
이름	설명
LoginResponse	Access Token, Refresh Token, 토큰 타입, 사용자 프로필

소스 코드

```
// auth/service/AuthService.java
    @Transactional
    public LoginResponse login(LoginRequest request) {
        User user = userRepository.findByLoginId(request.loginId())
                .orElseThrow(() -> new ApiException(HttpStatus.UNAUTHORIZED,
"INVALID_CREDENTIALS", "Invalid login credentials"));

        if (!"active".equals(user.getStatus())) {
            throw new ApiException(HttpStatus.FORBIDDEN, "USER_INACTIVE",
"Inactive user status");
        }

        if (user.getPasswordHash() == null ||
!passwordEncoder.matches(request.password(), user.getPasswordHash())) {
            throw new ApiException(HttpStatus.UNAUTHORIZED,
"INVALID_CREDENTIALS", "Invalid login credentials");
        }

        revokeAllRefreshTokens(user.getId());

        JwtTokenService.TokenPair tokenPair =
jwtTokenService.issueTokenPair(user);
        JwtTokenService.DecodedRefreshToken decodedRefreshToken =
jwtTokenService.parseAndValidateRefreshToken(tokenPair.refreshToken());
        String refreshTokenHash =
jwtTokenService.hashToken(tokenPair.refreshToken());

        RefreshToken refreshTokenEntity = RefreshToken.issue(user,
refreshTokenHash, decodedRefreshToken.expiresAt());
        refreshTokenRepository.save(refreshTokenEntity);
        jwtTokenService.storeRefreshTokenHash(refreshTokenHash, user.getId(),
decodedRefreshToken.expiresAt());

        return new LoginResponse(
```

```
                tokenPair.accessToken(),
                tokenPair.refreshToken(),
                "Bearer",
                UserProfileResponse.from(user, hasTodayCondition(user.getId()),
balanceCurrency(user.getId()), mediaUrlResolver)
        );
    }
```

클래스 명	RoutineService
함수 명	getRecommendations(Long userId)
함수 설명	온보딩 정보를 바탕으로 기본 루틴을 점수화하고 사용자에게 적합한 루틴 2개를 추천한다.
인수	
이름	설명
userId	추천을 요청한 로그인 사용자 ID
반환 값	
이름	설명
List<RoutineRecommendationResponse>	추천 루틴 요약, 추천 점수, 추천 사유 목록

소스 코드

```
// routine/service/RoutineService.java
    @Transactional(readOnly = true)
    public List<RoutineRecommendationResponse> getRecommendations(Long
userId) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"USER_NOT_FOUND", "User not found"));
        if (!user.isOnboardingCompleted()) {
            throw new ApiException(HttpStatus.BAD_REQUEST,
"ONBOARDING_REQUIRED", "온보딩 프로필을 먼저 저장해 주세요.");
        }

        return
routineRepository.findByDefaultRoutineTrueAndActiveTrueOrderByIdAsc().stream()
                .map(routine -> scoreRoutine(user, routine))

.sorted(Comparator.comparing(RoutineRecommendationResponse::score).reversed()
                        .thenComparing(response -> response.routine().id()))
                .limit(2)
                .toList();
    }
```

클래스 명	RoutineService

함수 명	createCustomRoutine(Long userId, CreateCustomRoutineRequest request)
함수 설명	사용자가 직접 입력한 세션과 운동 항목을 저장하여 커스텀 루틴을 생성한다.
인수	
이름	설명
userId	루틴을 생성하는 사용자 ID
request.name	루틴 이름
request.sessions	요일별 세션 및 운동 항목 목록
request.activate	생성 직후 활성 루틴으로 지정할지 여부
반환 값	
이름	설명
RoutineDetailResponse	생성된 루틴의 기본 정보, 세션, 운동 항목 상세

소스 코드

```
// routine/service/RoutineService.java
    @Transactional
    public RoutineDetailResponse createCustomRoutine(Long userId,
CreateCustomRoutineRequest request) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"USER_NOT_FOUND", "사용자를 찾을 수 없습니다."));

        int estimatedMinutes = estimatedMinutes(request.sessions());
        Routine routine = routineRepository.save(Routine.createCustom(
                user,
                request.name().trim(),
                normalizeText(request.description()),
                estimatedMinutes == 0 ? null : estimatedMinutes
        ));
        saveSessionsAndItems(routine, request.sessions());

        if (request.activate() == null || Boolean.TRUE.equals(request.activate())) {
            user.updateActiveRoutine(routine);
        }

        return routineDetail(routine.getId());
    }
```

클래스 명	ConditionService
함수 명	updateTodayCondition(Long userId, ConditionTodayRequest request)
함수 설명	오늘의 컨디션 입력값을 점수화하고 운동 배율을 계산하여 해당 날짜의

	컨디션 기록을 갱신한다.
인수	
이름	설명
userId	컨디션을 기록하는 사용자 ID
request.conditionLevel	사용자가 선택한 현재 컨디션 단계
request.sleepHours	수면 시간
request.stressLevel	스트레스 수준
반환 값	
이름	설명
ConditionLogResponse	저장된 컨디션 점수, 운동 배율, 기록 시각
소스 코드	

```
// condition/service/ConditionService.java
    @Transactional
    public ConditionLogResponse updateTodayCondition(Long userId,
ConditionTodayRequest request) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"USER_NOT_FOUND", "User not found"));

        LocalDate today = KoreanTime.today();
        BigDecimal conditionScore = calculateConditionScore(request);
        BigDecimal exerciseMultiplier =
calculateExerciseMultiplier(conditionScore);

        ConditionLog conditionLog =
conditionLogRepository.findByUser_IdAndLogDate(userId, today)
                .orElseGet(() -> ConditionLog.create(
                        user,
                        today,
                        request.conditionLevel(),
                        request.sleepScore(),
                        request.stressScore(),
                        fatigueScore(request),
                        request.energyLevel(),
                        conditionScore,
                        exerciseMultiplier
                ));

        conditionLog.updateScores(
                request.conditionLevel(),
                request.sleepScore(),
                request.stressScore(),
                fatigueScore(request),
```

```
                request.energyLevel(),
                conditionScore,
                exerciseMultiplier
        );

        ConditionLog savedConditionLog =
conditionLogRepository.save(conditionLog);
        return ConditionLogResponse.from(savedConditionLog);
    }
```

클래스 명	QuestService
함수 명	getTodayQuest(Long userId, boolean issueRecoveryIfCompleted)
함수 설명	만료된 퀘스트를 정리하고 오늘 수행할 퀘스트가 있으면 반환하며, 없으면 새 퀘스트를 생성한다.
인수	
이름	설명
userId	퀘스트를 조회하는 사용자 ID
issueRecoveryIfCompleted	완료 후 회복 퀘스트 추가 발급 여부
반환 값	
이름	설명
QuestResponse	오늘 퀘스트의 상태, 목표, 보상, 만료 정보
소스 코드	

```
// quest/service/QuestService.java
    @Transactional
    public QuestResponse getTodayQuest(Long userId, boolean
issueRecoveryIfCompleted) {
        LocalDate today = KoreanTime.today();
        userQuestRepository.expireUnfinishedBefore(userId, today);

        List<UserQuest> todayQuests =
userQuestRepository.findByUser_IdAndQuestDateOrderByCreatedAtAscIdAsc(userId,
today);
        if (!todayQuests.isEmpty()) {
            return returnExistingTodayQuest(todayQuests.get(0));
        }
        return createTodayQuest(userId, today);
    }
```

클래스 명	QuestService
함수 명	completeQuest(Long userId, Long questId, CompleteQuestRequest

	request)
함수 설명	사용자의 퀘스트 진행값과 건강 데이터를 검증해 완료 처리하고 보상과 업적을 지급한다.
인수	
이름	설명
userId	퀘스트 수행 사용자 ID
questId	완료 처리할 퀘스트 ID
request.progressValue	사용자가 제출한 수행량
request.healthSamples	검증에 사용할 건강 데이터 샘플
반환 값	
이름	설명
QuestResponse	완료 처리 후 퀘스트 상태와 지급된 보상 정보

소스 코드

```
// quest/service/QuestService.java
    @Transactional
    public QuestResponse completeQuest(Long userId, Long questId,
CompleteQuestRequest request) {
        UserQuest quest = userQuestRepository.findByIdAndUser_Id(questId,
userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"QUEST_NOT_FOUND", "퀘스트를 찾을 수 없습니다."));

        LocalDate today = KoreanTime.today();
        if (quest.getQuestDate().isBefore(today)) {
            quest.expire();
            throw new ApiException(HttpStatus.CONFLICT, "QUEST_EXPIRED",
"지난 날짜의 미완료 퀘스트는 완료할 수 없습니다.");
        }
        if (UserQuest.STATUS_EXPIRED.equals(quest.getStatus())) {
            throw new ApiException(HttpStatus.CONFLICT, "QUEST_EXPIRED",
"만료된 퀘스트는 완료할 수 없습니다.");
        }
        boolean newlyCompleted =
!UserQuest.STATUS_COMPLETED.equals(quest.getStatus());
        if (newlyCompleted) {
            HealthQuestProgress healthProgress = evaluateHealthProgress(quest,
request);
            QuestCompletion completion = completionFor(quest, request,
healthProgress);
            Integer progressValue = healthProgress != null &&
healthProgress.progressValue() != null
                    ? healthProgress.progressValue()
```

```
                : request == null ? null : request.progressValue();
            quest.complete(progressValue, completion.proof());
            rewardService.issueQuestCompletionRewards(quest,
completion.rewardExp(), completion.rewardCurrency(),
completion.rewardMemoPrefix());
            achievementBadgeService.awardForQuestCompletion(quest);
        }
        return QuestResponse.from(quest, exercisesFromContext(quest));
    }
```

클래스 명	HealthDataService
함수 명	summarize(Long userId, HealthDataSyncRequest request)
함수 설명	모바일에서 전달된 건강 데이터 샘플을 표준 형태로 변환하고 일별 요약 데이터에 반영한다.
인수	
이름	설명
userId	건강 데이터를 동기화하는 사용자 ID
request.samples	걸음 수, 거리, 칼로리, 운동 세션 등의 원본 샘플 목록
반환 값	
이름	설명
HealthDataSyncResponse	처리된 샘플 수, 타입별 개수, 건강 지표 요약
소스 코드	

```
// health/service/HealthDataService.java
    @Transactional
    public HealthDataSyncResponse summarize(Long userId,
HealthDataSyncRequest request) {
        List<NormalizedHealthSample> samples = normalizeSamples(request ==
null ? null : request.samples());
        saveDailySummaries(userId, samples);
        return responseFrom(samples);
    }
```

클래스 명	ShopService
함수 명	purchaseItem(Long userId, Long itemId, PurchaseItemRequest request)
함수 설명	상점 아이템 구매 가능 여부와 지갑 잔액을 검증한 뒤 인벤토리와 거래 내역을 갱신한다.
인수	
이름	설명
userId	구매하는 사용자 ID

itemId	구매할 상점 아이템 ID
request.quantity	구매 수량
반환 값	
이름	설명
ShopPurchaseResponse	구매 결과, 아이템 정보, 변경된 잔액, 거래 내역

소스 코드

```
// shop/service/ShopService.java
    @Transactional
    public ShopPurchaseResponse purchaseItem(Long userId, Long itemId,
PurchaseItemRequest request) {
        User user = userRepository.findById(userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"USER_NOT_FOUND", "사용자를 찾을 수 없습니다."));
        Item item = itemRepository.findByIdAndActiveTrue(itemId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"ITEM_NOT_FOUND", "구매할 아이템을 찾을 수 없습니다."));
        if (!Boolean.TRUE.equals(item.getSellable())) {
            throw new ApiException(HttpStatus.BAD_REQUEST,
"ITEM_NOT_SELLABLE", "판매 중인 아이템이 아닙니다.");
        }

        int quantity = request.quantity();
        int totalPrice = totalPrice(item, quantity);
        Wallet wallet = walletRepository.findByUserId(userId)
                .orElseGet(() -> walletRepository.save(Wallet.create(user)));
        if (wallet.getBalanceCurrency() < totalPrice) {
            throw new ApiException(HttpStatus.BAD_REQUEST,
"INSUFFICIENT_BALANCE", "골드가 부족합니다.");
        }

        wallet.debit(totalPrice);
        UserItem userItem = userItemRepository.findByUser_IdAndItem_Id(userId,
itemId)
                .orElseGet(() -> UserItem.create(user, item, 0));
        userItem.increaseQuantity(quantity);
        UserItem savedUserItem = userItemRepository.save(userItem);
        walletTransactionRepository.save(WalletTransaction.purchase(user,
totalPrice, item.getId(), "아이템 구매"));

        return new ShopPurchaseResponse(
                PurchasedItemResponse.from(savedUserItem),
                wallet.getBalanceCurrency(),
```

```
                new
WalletTransactionResponse(WalletTransaction.TX_TYPE_PURCHASE, -totalPrice)
        );
    }
```

클래스 명	BattleService
함수 명	match(Long userId, BattleMode mode)
함수 설명	현재 배틀과 대기열을 확인하고, 상대가 있으면 배틀을 생성하며 없으면 매칭 대기 상태를 생성한다.
인수	
이름	설명
userId	배틀 매칭을 요청한 사용자 ID
mode	일간 또는 주간 배틀 모드
반환 값	
이름	설명
BattleDetailResponse	매칭된 배틀 상세 또는 대기열 상태 정보
소스 코드	

```
// battle/service/BattleService.java
    @Transactional
    public BattleDetailResponse match(Long userId, BattleMode mode) {
        User currentUser = userRepository.findById(userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"USER_NOT_FOUND", "사용자를 찾을 수 없습니다."));
        Instant now = KoreanTime.nowInstant();
        BattlePeriod queuePeriod = BattlePeriod.fromStart(mode, now);

        Optional<Battle> currentBattle = findCurrentBattle(userId, mode, now);
        if (currentBattle.isPresent()) {
            return toDetail(currentBattle.get(), userId);
        }

        Optional<BattleMatchQueue> currentWaitingQueue =
battleMatchQueueRepository
                .findByUser_IdAndModeAndStatusAndExpiresAtAfter(
                        userId,
                        mode,
                        BattleMatchQueueStatus.WAITING,
                        now
                );
        if (currentWaitingQueue.isPresent()) {
            return toWaitingDetail(currentUser,
```

```
BattlePeriod.fromQueue(currentWaitingQueue.get(), currentWaitingQueue.get());
        }

        Optional<BattleMatchQueue> opponentQueue =
battleMatchQueueRepository.findWaitingOpponents(
                userId,
                mode,
                now,
                PageRequest.of(0, 1)
        ).stream().findFirst();

        BattleMatchQueue myQueue =
battleMatchQueueRepository.save(BattleMatchQueue.waiting(
                currentUser,
                mode,
                queuePeriod.startDate(),
                queuePeriod.endDate(),
                queuePeriod.endsAt()
        ));
        if (opponentQueue.isEmpty()) {
            return toWaitingDetail(currentUser, queuePeriod, myQueue);
        }

        User opponent = opponentQueue.get().getUser();
        Instant matchedAt = KoreanTime.nowInstant();
        BattlePeriod battlePeriod = BattlePeriod.fromStart(mode, matchedAt);
        Battle battle = battleRepository.save(Battle.create(
                mode,
                battlePeriod.startDate(),
                battlePeriod.endDate(),
                matchedAt,
                battlePeriod.endsAt()
        ));
        List<BattleParticipant> participants =
battleParticipantRepository.saveAll(List.of(
                BattleParticipant.join(battle, currentUser),
                BattleParticipant.join(battle, opponent)
        ));
        participants.forEach(participant -> updateBaseline(participant,
battlePeriod));
        opponentQueue.get().match(battle, matchedAt);
```

```
        myQueue.match(battle, matchedAt);
        notificationService.sendBattleMatched(battle, participants);
        return toDetail(battle, userId);
    }
```

클래스 명	BattleService
함수 명	getResult(Long userId, Long battleId)
함수 설명	배틀 종료 시간이 지난 경우 결과를 확정하고 사용자 기준의 배틀 결과를 반환한다.
인수	
이름	설명
userId	결과를 조회하는 사용자 ID
battleId	결과를 확정하거나 조회할 배틀 ID
반환 값	
이름	설명
BattleResultResponse	사용자와 상대의 점수, 승패, 보상 결과

소스 코드

```
// battle/service/BattleService.java
    @Transactional
    public BattleResultResponse getResult(Long userId, Long battleId) {
        Battle battle = battleRepository.findByIdAndParticipantUserId(battleId,
userId)
                .orElseThrow(() -> new ApiException(HttpStatus.NOT_FOUND,
"BATTLE_NOT_FOUND", "배틀을 찾을 수 없습니다."));
        if (!battle.isFinalized() &&
!KoreanTime.nowInstant().isBefore(battle.getEndsAt())) {
            finalizeBattle(battle);
        }
        return toResult(battle, userId);
    }
```

클래스 명	AiProxyService
함수 명	forward(HttpMethod method, String path, String requestBody, String authorizationHeader)
함수 설명	Spring 서버가 AI 서버의 API를 대신 호출하고 응답 또는 오류 메시지를 공통 형식으로 변환한다.
인수	
이름	설명
method	GET 또는 POST 요청 방식
path	AI 서버에 전달할 API 경로
requestBody	POST 요청 본문 JSON 문자열

authorizationHeader	인증이 필요한 요청에 전달할 Authorization 헤더
반환 값	
이름	설명
Object	AI 서버 JSON 응답 또는 원문 응답을 파싱한 객체

소스 코드

```
// ai/service/AiProxyService.java

    private Object forward(HttpMethod method, String path, String requestBody,
String authorizationHeader) {
        RequestBodySpec requestSpec = restClient.method(method)
                .uri(path)
                .accept(MediaType.APPLICATION_JSON)
                .contentType(MediaType.APPLICATION_JSON);

        if (authorizationHeader != null && !authorizationHeader.isBlank()) {
            requestSpec.header(HttpHeaders.AUTHORIZATION,
authorizationHeader);
        }

        if (requestBody != null) {
            requestSpec.body(requestBody);
        }

        return requestSpec.exchange((request, response) -> {
            byte[] bodyBytes = response.getBody().readAllBytes();
            Object responseBody = parseBody(bodyBytes);

            if (response.getStatusCode().isError()) {
                String message = extractErrorMessage(responseBody);
                HttpStatus status =
HttpStatus.resolve(response.getStatusCode().value());
                throw new ApiException(
                        status == null ? HttpStatus.BAD_GATEWAY : status,
                        "AI_SERVER_ERROR",
                        message
                );
            }

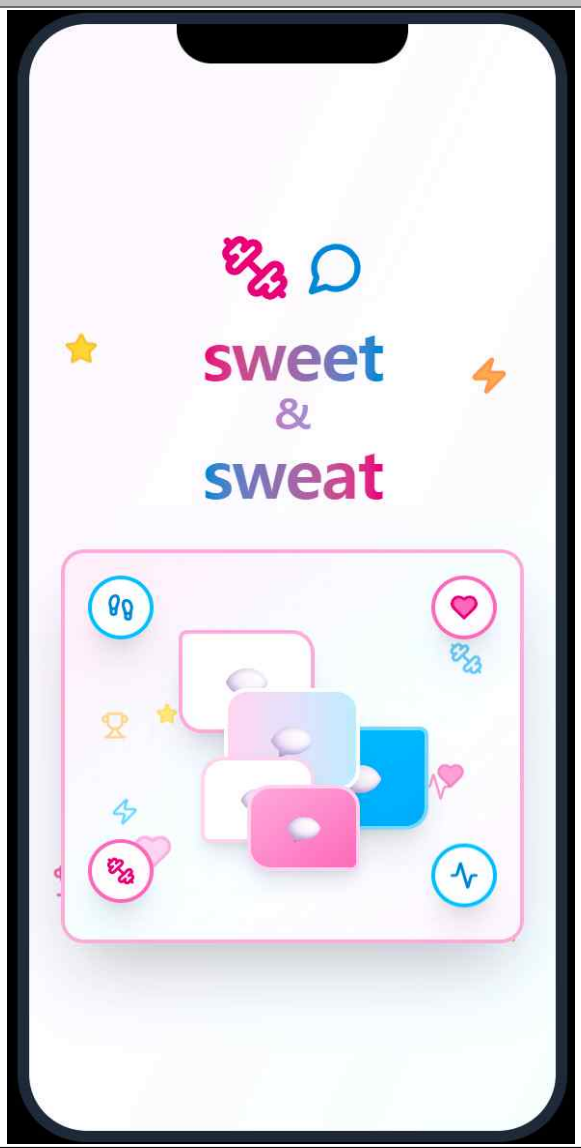
            return responseBody;
        });
    }
```

9. 사용자 매뉴얼

9.1 사용자 매뉴얼

인터페이스 식별자	UI-01

모바일



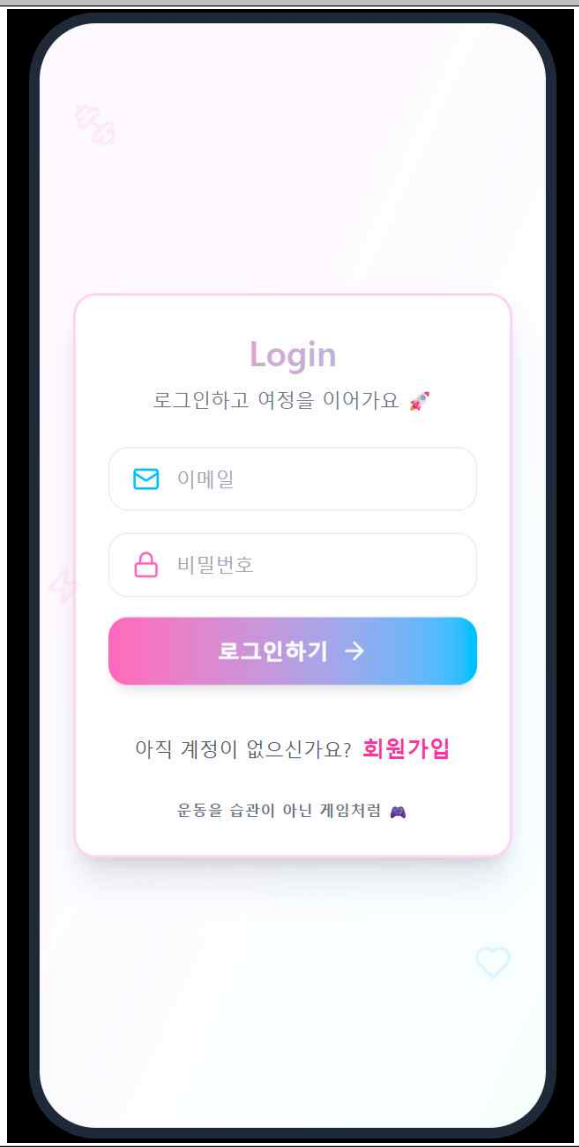
설명

서비스 시작 화면

메인 기능으로 진입하기 전 초기 로딩 또는 인트로 화면 역할을 한다.

인터페이스 식별자	UI-02

모바일



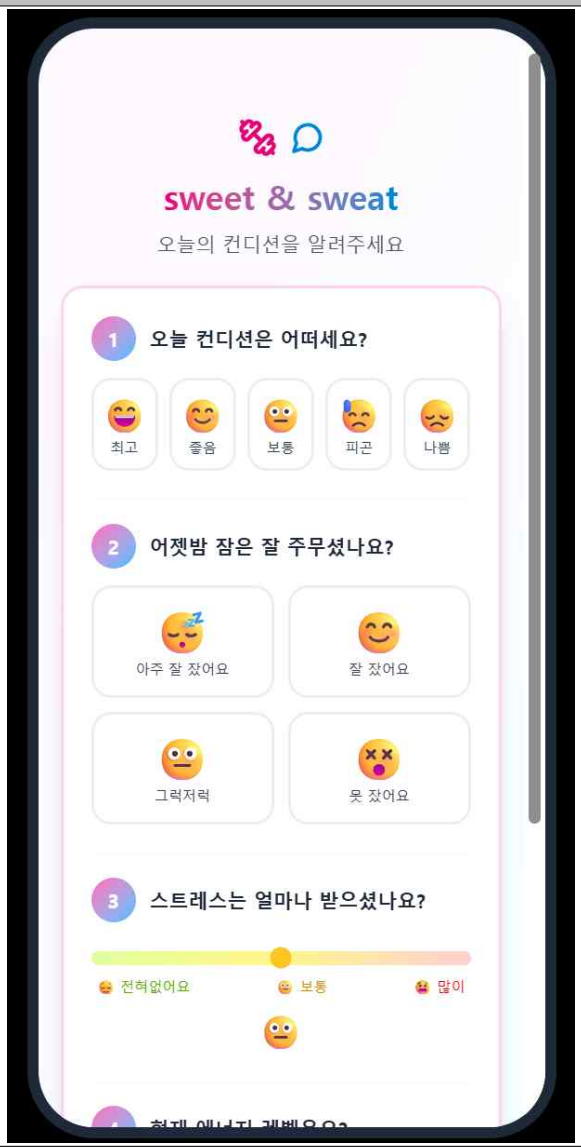
설명

로그인 화면

사용자가 로그인할 수 있는 화면이다.

인터페이스 식별자	UI-03

모바일



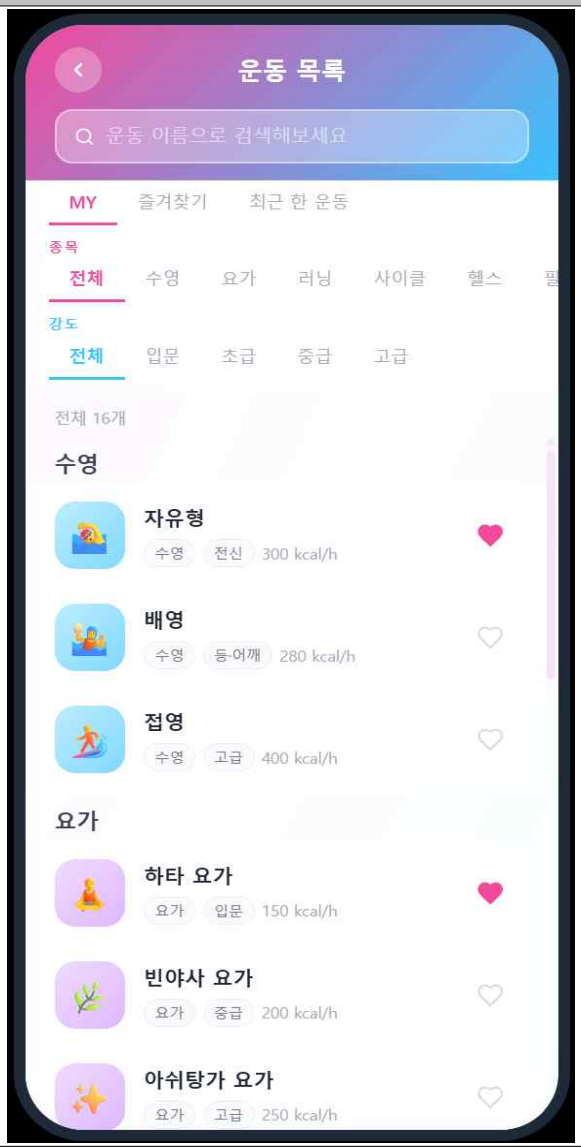
설명

컨디션 입력 화면

사용자가 오늘의 컨디션, 수면 상태, 스트레스 정도 등을 입력하여 이후 루틴 강도 조절 또는 퀘스트 진행에 반영할 수 있도록 한다.

인터페이스 식별자	UI-04

모바일



설명

운동 목록 조회 화면

사용자가 운동 종목을 검색하고 즐겨찾기, 운동 종류, 강도 기준으로 필터링하여 원하는 운동을 탐색할 수 있도록 구성된다.

인터페이스 식별자	UI-05

모바일



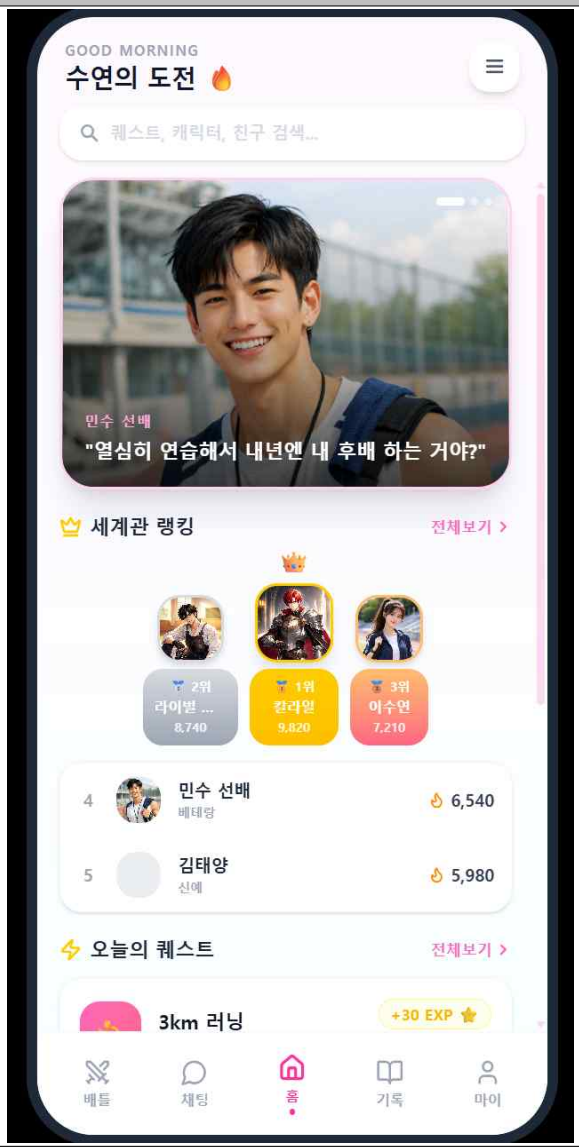
설명

컨디션 통계 조회 화면

사용자가 주간, 월간, 연간 단위로 자신의 컨디션 변화와
운동 효과(소모 칼로리, 걸음 수 등)를 확인할 수 있도록 구성된 화면이다.

인터페이스 식별자	UI-06

모바일



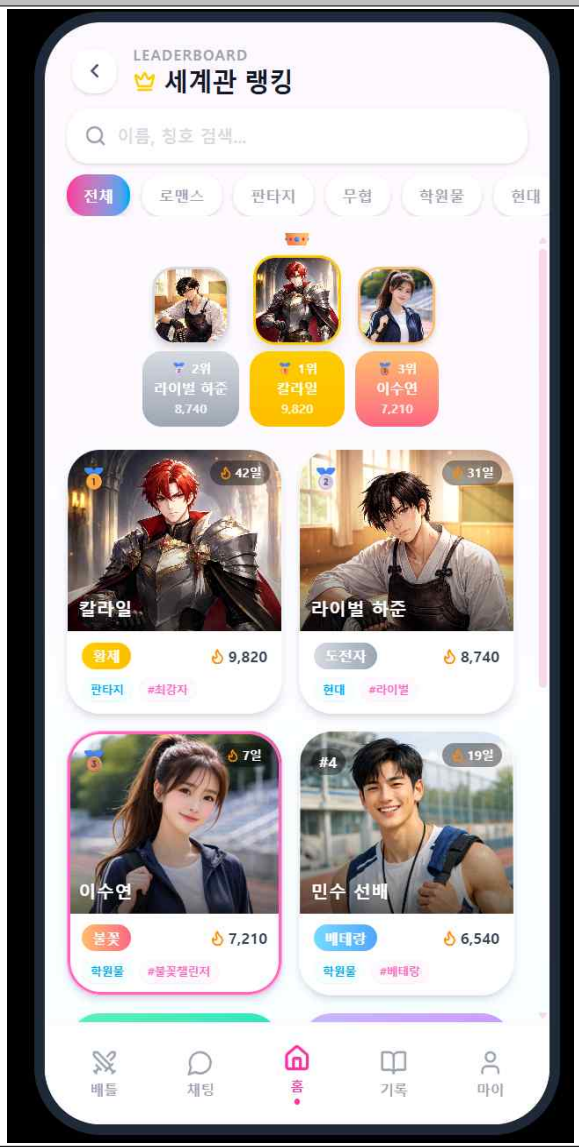
설명

홈 화면

세계관 랭킹, 오늘의 퀘스트, 주간 랭킹 등을 확인할 수 있는 화면이다.
하단의 네비게이션 바를 통해 다른 화면으로 이동할 수 있다.

인터페이스 식별자	UI-07

모바일



설명

세계관 랭킹 화면

세계관 랭킹을 확인할 수 있는 화면이다.

상단의 필터를 통해 장르별 인기순위를 확인할 수 있다.

인터페이스 식별자	UI-08

모바일



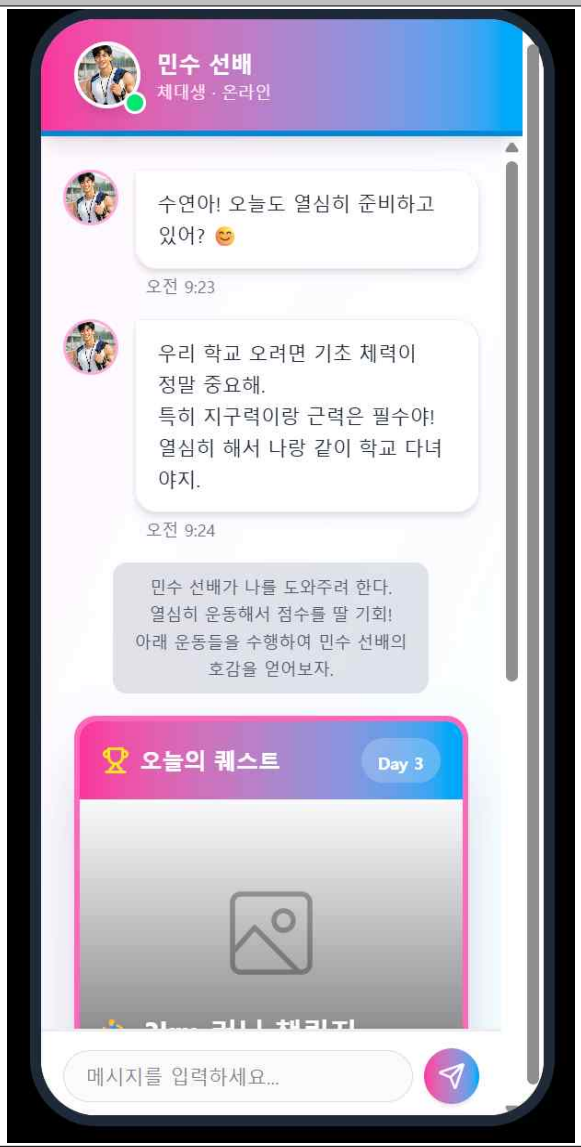
설명

캐릭터 선택 화면

사용자가 캐릭터를 선택하고, 이름을 작성할 수 있는 화면이다.

인터페이스 식별자	UI-09

모바일



설명

캐릭터 대화 및 오늘의 퀘스트 화면

사용자는 캐릭터와 대화를 진행하며 운동 동기를 부여받고,
하단에서 오늘의 퀘스트를 확인할 수 있다.

인터페이스 식별자	UI-10

모바일



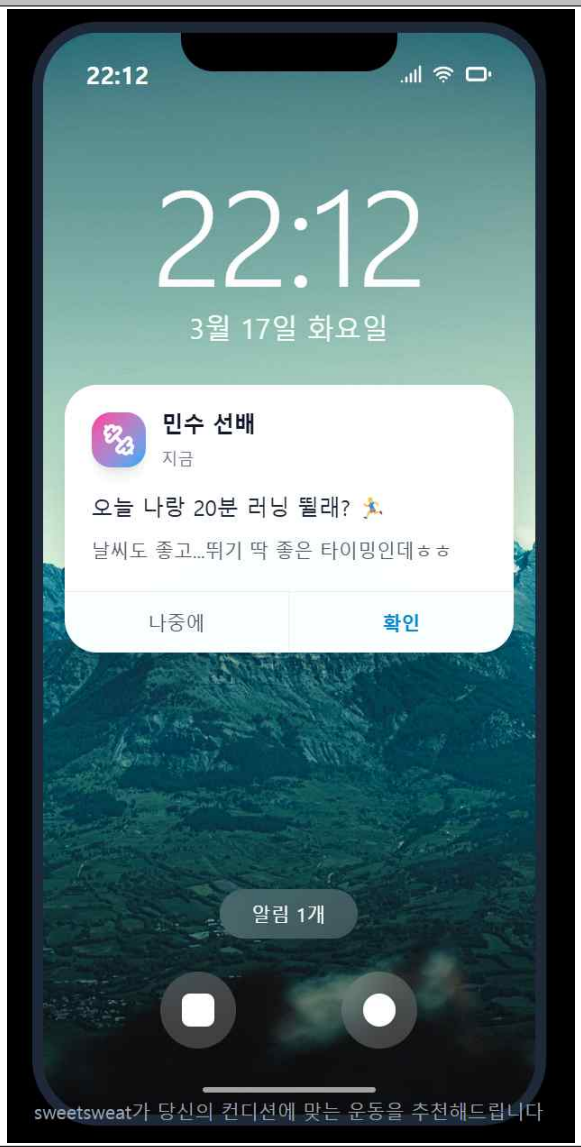
설명

스토리 이벤트 발생 화면

캐릭터가 특별 제안이나 이벤트를 제시하고, 사용자는 참여 여부를 선택할 수 있다.

인터페이스 식별자	UI-11

모바일



설명

푸시 알림 화면

캐릭터 또는 시스템이 사용자에게 운동 참여를 유도하는 알림을 전달하며, 사용자는 알림을 확인하고 바로 운동이나 퀘스트 흐름으로 진입할 수 있다.

인터페이스 식별자	UI-12

모바일



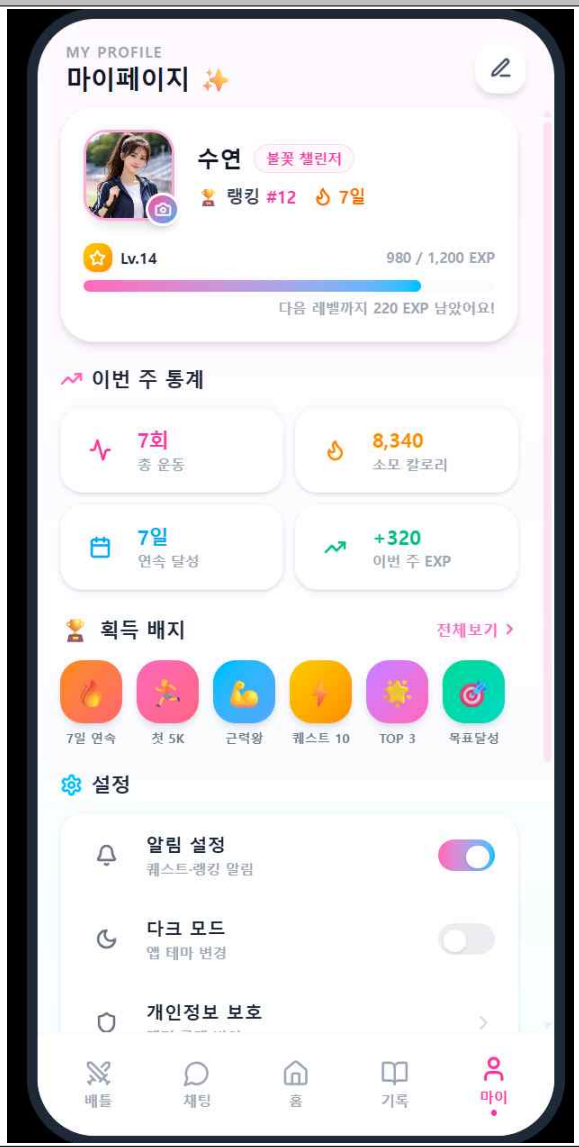
설명

배틀 화면

다른 사용자와 주간, 일간 등 기간을 설정하여 운동 기록으로
경쟁할 수 있는 화면이다..

인터페이스 식별자	UI-13

모바일



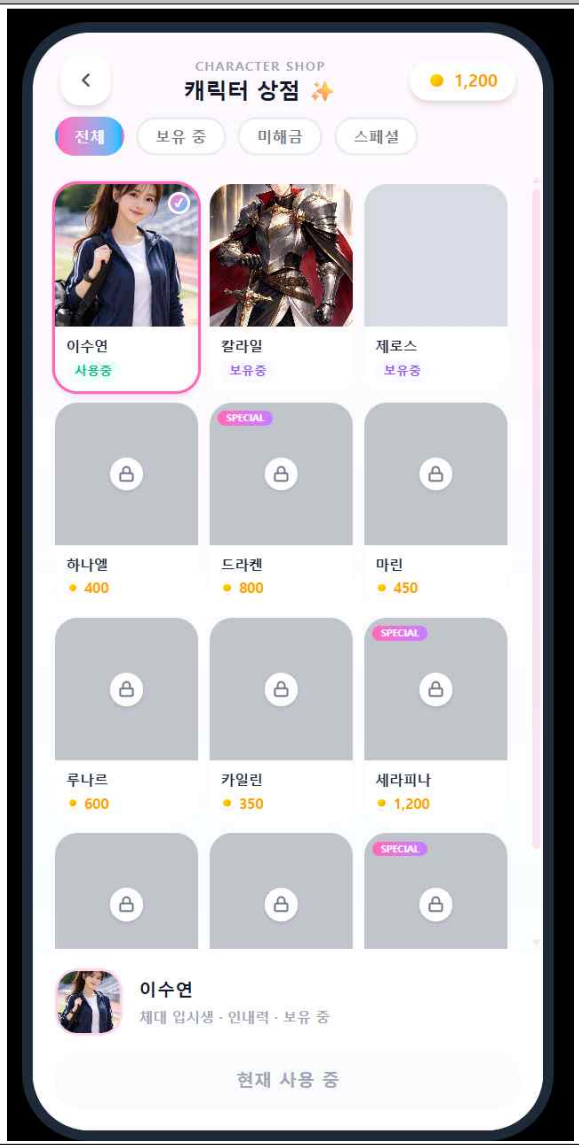
설명

마이페이지 화면

운동 기록, 획득 배지, 개인정보 등을 확인할 수 있는 화면이다.

인터페이스 식별자	UI-14

모바일



설명

캐릭터 상점 화면

공략을 완료한 캐릭터 중 원하는 캐릭터를 구매할 수 있는 화면이다.
보유 중, 해금 여부, 스페셜에 따라 구분하여 볼 수 있다.

10. 역할분담

구분	성명	담당 역할
팀장	하대영	백엔드 API 설계 및 구현 DB 구조 설계와 Spring Boot 서버 개발 인증/JWT, 루틴, 컨디션, 퀘스트, 건강 데이터, 상점, 배틀, 보상 기능 구현 AI 서버 연동 프록시 및 개발/배포 환경 관리
팀원	노가영	AI 서버 설계 및 구현 FastAPI, LangChain, LangGraph 기반 스토리 생성 및 플레이 API 개발 캐릭터 대화, 선택지 생성, 스토리 퀘스트 연동 로직 구현 AI 서버 PostgreSQL 데이터 관리 및 백엔드/모바일 연동 지원
팀원	함서현	React Native 모바일 앱 화면 및 UI/UX 구현 홈, 퀘스트, 캐릭터, 랭킹, 배틀, 마이페이지 등 주요 화면 개발 Health Connect 기반 건강 데이터 연동과 FCM/알림 기능 연동 모바일 앱 API 연동 테스트 및 사용자 흐름 검증

11. 비고

이름	소감
함서현	Sweet & Sweat 프로젝트는 캐릭터챗 서비스에 대한 관심에서 시작되었습니다. 이전에 캐릭터챗 서비스를 만드는 회사의 면접을 본 적이 있었는데, 그 과정에서 캐릭터챗 시장이 생각보다 훨씬 크다는 것을 알게 되었습니다. 동시에 면접 당시에는 캐릭터챗 서비스가 어떤 방식으로 기획되고, 어떤 기술로 구현되며, 사용자의 몰입을 어떻게 유지하는지에 대해 충분히 답하지 못한 부분도 있었습니다. 그 경험이 아쉬움으로 남았고, 이후 직접 만들어 보면서 이해해 보고 싶다는 생각을 하게 되었습니다. 그래서 팀원들에게 캐릭터, 스토리, 대화형 AI를 활용한 프로젝트를 제안했고, 여기에 단순한 재미를 넘어 사회적으로도 의미 있는 주제를 담고 싶어 운동이라는 요소를 결합했습니다. 그렇게 Sweet & Sweat는 캐릭터와 세계관을 통해 사용자가 운동을 지속하도록 돕는 서비스로 방향을 잡게 되었습니다. 프로젝트를 진행하면서 가장 뿌듯했던 점은, 과거 면접에서 제대로 설명하지 못했던 부분들을 실제 개발 과정을 통해 하나씩 이해하게 되었다는 점입니다. 캐릭터챗은 단순히 AI가 대답을 잘하는 기능만으로 완성되는 것이 아니라, 사용자의 선택, 캐릭터의 성격, 스토리 흐름, 이전 기록, 보상 구조가 함께 연결되어야 몰입감이 생긴다는 것을 알게 되었습니다. 또한 운동 앱 역시 단순히 걸음 수나 운동 시간을 보여주는 것만으로는 사용자가 꾸준히 사용하기 어렵고, 목표를 잘게 나누고 즉각적인 피드백과 보상을 제공해야 지속성이 생긴다는 점을 느꼈습니다. 처음에는 흥미로운

	아이디어에서 출발했지만, 개발을 진행할수록 이 프로젝트가 사용자의 행동 변화를 설계하는 과정이라는 생각이 들었습니다. 모바일 개발자로서도 많은 성장을 경험했습니다. React Native로 화면을 구현하면서 사용자가 실제로 앱을 사용할 때 어떤 흐름이 자연스러운지 계속 고민하게 되었고, 홈 화면에서 오늘의 퀘스트를 확인한 뒤 캐릭터 상태, 보상, 랭킹으로 이어지는 과정이 끊기지 않도록 구성하는 일이 중요하다는 것을 느꼈습니다. 단순히 화면을 예쁘게 만드는 것이 아니라, 사용자가 다음에 무엇을 해야 하는지 바로 이해할 수 있도록 설계하는 것이 모바일 앱 개발의 핵심이라는 점을 체감했습니다. 특히 AI를 통해 스토리나 퀘스트가 생성되는 과정에서 사용자가 잠시 대기해야 하는 순간이 발생했는데, 이 시간을 단순한 로딩 화면으로 두면 지루함을 느끼거나 앱이 멈춘 것처럼 받아들일 수 있다고 생각했습니다. 그래서 대기 시간 동안 애니메이션이나 진행 상황을 보여 주는 요소를 넣으면 사용자가 현재 어떤 과정이 진행되고 있는지 자연스럽게 이해할 수 있겠다는 것을 깨달았습니다. 이를 통해 개발자는 기능이 작동하는지만 확인하는 것이 아니라, 사용자가 그 기능을 어떻게 느끼는지까지 고려해야 한다는 점을 배웠습니다. 또한 저는 테스트를 전담하면서 개발된 기능이 실제 사용자 환경에서 안정적으로 동작하는지 확인했습니다. 회원가입, 로그인, 퀘스트 조회, 캐릭터 선택, 랭킹 조회, 상점 이용 등 주요 기능을 반복적으로 테스트했고, React Native 앱과 Spring Boot 서버 간 API 요청과 응답이 올바르게 이어지는지도 검증했습니다. 오류를 발견하면 원인을 정리해 팀원들과 공유하고, 수정 후 다시 확인하는 과정을 거치며 개발은 코드 작성에서 끝나는 것이 아니라 안정적으로 동작하는지 검증하는 과정까지 포함된다는 것을 배웠습니다. 더불어 웨어러블 기반 건강 데이터 연동을 다루며 건강 데이터는 단순히 값을 받아오는 문제가 아니라 기기 지원 여부, 권한 설정, OS 버전, 연동 방식 등 다양한 조건의 영향을 받는다는 것을 알게 되었고, 이를 통해 모바일 개발자는 앱 내부 코드뿐만 아니라 사용자의 실제 기기 환경까지 함께 고려해야 한다는 점을 배웠습니다. Sweet & Sweat를 통해 저는 좋은 앱은 기능을 많이 넣는 것이 아니라, 사용자가 계속 사용하고 싶게 만드는 흐름을 설계하는 것이라는 점을 배웠습니다. 처음에는 아쉬움으로 시작한 프로젝트였지만, 이제는 캐릭터 기반 서비스와 운동 게이미피케이션의 구조를 직접 설명할 수 있게 되었다는 점에서 큰 성취감을 느낍니다. 앞으로도 단순히 기술을 구현하는 개발자가 아니라, 사용자의 문제와 감정을 이해하고, 더 편리한 경험을 설계하는 개발자로 성장하고 싶습니다.

이름	소감
노가영	이번 프로젝트를 진행하면서 단순히 기능을 구현하는 것보다, 여러 서버와 기능이 자연스럽게 연결되도록 설계하는 과정이 훨씬 중요하다는 것을 느꼈다. AI 서버, Spring Boot 서버, 프론트엔드 앱, 운동 루틴 API가 각각 따로 동작하는 것이 아니라 하나의 서비스 흐름 안에서 맞물려야 했기 때문에 API 응답 구조와

	데이터 저장 방식의 중요성을 크게 체감했다. 특히 LangGraph와 LangChain을 활용해 스토리 진행, 캐릭터 대화, 선택지 생성, 운동 퀘스트 변환 기능을 구현하면서 AI 기능은 단순히 문장을 생성하는 것이 아니라 사용자의 현재 상황과 이전 대화 맥락을 유지하는 구조가 필요하다는 점을 배웠다. 프롬프트를 조금만 잘못 설계해도 캐릭터 대사가 어색해지거나 스토리 전개가 꼬일 수 있었고, 이를 해결하기 위해 프롬프트뿐 아니라 검수 로직과 데이터 흐름까지 함께 조정해야 했다. 또한 프론트엔드와 협업하면서 백엔드 응답이 실제 화면에서 어떻게 사용되는지 고려하는 습관을 기를 수 있었다. 처음에는 필요한 데이터를 많이 내려주는 것이 좋다고 생각했지만, 실제로는 프론트에서 필요한 필드만 명확하게 제공하고 불필요한 내부 상태나 프롬프트 정보는 숨기는 것이 더 중요하다는 것을 알게 되었다. 운동 루틴을 스토리 퀘스트로 변환하는 기능을 만들며 서비스의 핵심 아이디어를 더 구체적으로 이해할 수 있었다. 단순한 운동 기록이 아니라, 사용자가 스토리 속 미션을 수행하듯 운동을 하게 만드는 방식은 Sweet & Sweat만의 차별점이라고 느꼈다. 이 과정에서 사용자의 운동 데이터와 AI 생성 콘텐츠가 결합될 때 더 몰입감 있는 경험을 만들 수 있다는 가능성을 확인했다. 이번 프로젝트를 통해 FastAPI, PostgreSQL, LangChain, LangGraph, Git 협업 등 다양한 기술을 실제 서비스 흐름 안에서 사용해볼 수 있었고, 오류를 수정하고 팀원들과 데이터를 맞추는 과정에서 실무적인 협업 경험도 쌓을 수 있었다. 완벽하게 원하는 결과를 한 번에 만들기는 어려웠지만, 문제를 하나씩 분석하고 수정하면서 AI 기능을 서비스에 적용하는 방식에 대해 많이 배울 수 있었다.

이름	소감
하대영	Sweet & Sweat 프로젝트를 진행하면서 가장 크게 느낀 점은 하나의 서비스가 완성되기 위해서는 화면, 서버, AI, 데이터베이스가 각각 잘 동작하는 것만으로는 부족하다는 점이다. 사용자가 앱에서 버튼을 누르는 순간부터 백엔드 API가 요청을 처리하고, 데이터베이스에 상태가 저장되며, AI 서버와 모바일 화면까지 다시 자연스럽게 이어져야 실제로 사용할 수 있는 서비스가 된다는 것을 체감했다. 처음에는 기능을 하나씩 구현하는 데 집중했지만, 프로젝트가 진행될수록 각 기능이 서로 어떤 데이터를 주고받고 어떤 예외 상황을 처리해야 하는지 설계하는 일이 더 중요하다는 것을 알게 되었다. 저는 주로 Spring Boot 기반 백엔드 서버와 데이터베이스 구조, API 연동 흐름을 담당했다. 회원가입과 로그인, JWT 기반 인증, 온보딩 프로필, 루틴 추천과 커스텀 루틴, 컨디션 기록, 오늘의 퀘스트 생성과 완료 처리, 보상 지급, 상점 구매, 배틀 매칭과 결과 산정, 건강 데이터 동기화 등 여러 도메인을 구현하면서 서비스 내부의 데이터 흐름을 일관되게 유지하는 것이 쉽지 않다는 것을 배웠다. 특히 퀘스트와 배

	틀 기능은 단순 CRUD가 아니라 사용자 상태, 건강 데이터, 보상, 알림, 만료 처리 등이 함께 연결되어야 했기 때문에 트랜잭션 범위와 예외 처리를 더 신중하게 고민하게 되었다. AI 서버와 모바일 앱을 백엔드와 연결하는 과정에서도 많은 것을 배웠다. AI 서버가 생성한 스토리와 캐릭터 정보가 모바일 화면에서 자연스럽게 보이려면 응답 구조가 명확해야 했고, 모바일 앱에서 Health Connect를 통해 수집한 건강 데이터가 퀘스트와 배틀 검증에 사용되려면 백엔드에서 공통 지표로 정규화해야 했다. 이 과정에서 API는 단순히 데이터를 주고받는 통로가 아니라 팀원들이 서로 협업하기 위한 약속이라는 점을 느꼈다. 필드 이름 하나, 상태 코드 하나가 프론트엔드와 AI 서버의 구현 방식에 영향을 줄 수 있다는 점을 경험하며 문서화와 응답 구조 정리의 중요성도 배웠다. 프로젝트를 진행하며 어려웠던 부분은 기능이 많아질수록 테스트해야 할 조합도 함께 늘어난다는 점이었다. 로그인 상태, 활성 루틴 여부, 오늘의 컨디션 입력 여부, 건강 데이터 동기화 여부, 퀘스트 완료 상태, 배틀 기간 등 다양한 조건에 따라 결과가 달라졌기 때문에 단순히 정상 케이스만 확인해서는 부족했다. 오류가 발생했을 때는 데이터베이스 상태와 API 응답, 모바일 화면 흐름을 함께 확인해야 했고, 이를 반복하면서 문제를 추측하기보다 실제 데이터와 로그를 기준으로 원인을 찾는 습관을 기를 수 있었다. 이번 프로젝트를 통해 백엔드 개발자는 API를 구현하는 사람을 넘어 서비스 전체 흐름을 안정적으로 연결하는 역할을 해야 한다는 것을 배웠다. 팀장으로서 기능을 나누고 문서를 정리하며 팀원들의 AI 서버와 모바일 앱 작업이 백엔드와 맞물리도록 조율하는 과정도 의미 있는 경험이었다. 완성된 결과물을 보며 아쉬운 부분도 있었지만, 사용자 인증부터 운동 퀘스트, AI 스토리, 보상, 배틀까지 하나의 흐름으로 연결되는 서비스를 직접 만들어 보았다는 점에서 큰 성취감을 느꼈다. 앞으로도 단순히 동작하는 코드를 작성하는 데서 그치지 않고, 사용자가 실제로 안정적이고 자연스럽게 사용할 수 있는 시스템을 설계하는 개발자로 성장하고 싶다.

12. 회의록

회의명	주제 제안서 검토 회의		
일 시	1월 30일	장 소	비대면
의 안	MindFit Map 서비스 주제 제안 및 핵심 기능 검토		
토의내용	건강 데이터 기반 상태 분석과 지역 맞춤 운동 장소 추천을 핵심 아이디어로 정리하였다. MVP는 건강 데이터 수집, 스트레스/피로도 분석, 공공 체육시설 추천, 날씨 기반 필터링으로 범위를 좁히기로 하였다. 지역 커뮤니티와 감정 변화 추적은 차별화 기능으로 유지하되, 고도화 단계에서 확장하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

회의명	시스템 정의 및 데이터 활용 방안 회의		
일 시	1월 31일	장 소	비대면
의 안	운동 추천 범위, 장소 추천 방식, 활용 데이터 구체화		
토의내용	건강 데이터만으로 세부 부위 운동을 바로 추천하기는 어렵다고 판단하고, 상태 분석과 운동/장소 추천을 분리하기로 하였다. 장소 추천은 딥러닝 대신 거리, 날씨, 미세먼지, 영업시간, 컨디션 적합도를 반영한 rule-based 점수화 방식으로 정리하였다. 공공 체육시설 데이터와 지도 API를 함께 활용하고, 운동별 칼로리/처방 데이터는 운동 추천 근거로 사용하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유
	하대영 노가영 함서현		-

			
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

회의명	추천 구조 및 서비스 방향 회의		
일 시	2월 3일	장 소	비대면
의 안	레벨 기반 운동 추천 구조와 게이미피케이션 방향 검토		
토의내용	운동 추천은 딥러닝보다 건강 상태, 운동 속성, 환경 조건을 단계적으로 해석하는 레벨 기반 구조로 설계하기로 하였다. 운동 종목을 강도, 충격도, 제어가능성, 실내/실외 여부 등 속성으로 태깅하고, 사용자의 수면·심박·피로 상태에 따라 추천/주의/비추천 라벨을 부여하기로 하였다. 웨어러블이 없는 사용자를 고려해 직접 입력, GPS 체크인, 사진 인증 등을 보조 인증 방식으로 검토하였다. 추가 아이디어로 동네 운동시설 도장깨기, 레벨/경험치/칭호, 시설별 커뮤니티, AI 퀘스트를 결합한 FitQuest Map 방향을 논의하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유
	하대영 노가영 함서현		-

		함서현	
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

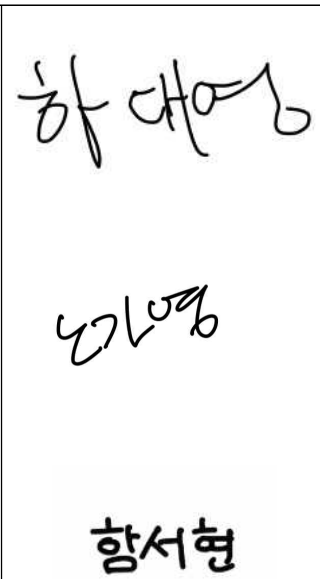
회의명	기능 범위 조정 및 게이미피케이션 도입 회의		
일 시	2월 6일	장 소	비대면
의 안	MindFit Map 기존 기능과 AI 게이미피케이션 추가안 검토		
토의내용	기존안은 건강 데이터 연동, 컨디션 분석, 운동/장소 추천, 운동 후 기록과 리포트를 중심 기능으로 유지하기로 하였다. 추가안으로 시설 도장깨기, 방문 횟수 기록, 경험치/레벨, 배지, AI 일일 퀘스트를 도입하되 구현 부담을 줄이기 위해 경량화하기로 하였다. 체크인은 GPS 기반 방문 인증을 우선 적용하고, 사진 인증·QR 인증·지역 랭킹·친구 기능·시즌 이벤트는 후순위 또는 제외 대상으로 정리하였다. AI 코치와 퀘스트 메시지는 통합하여 사용자 상태, 시설 정보, 운동 지식을 바탕으로 자연어 추천을 제공하는 방향으로 정리하였다. 최종 방향은 정신건강 리프레시 목적을 유지하면서, 실제 운동 장소 탐험과 퀘스트 보상을 결합해 지속 사용 동기를 높이는 서비스로 조정하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유
	하대영 노가영 함서현	하대영 노가영	-

		함서현	
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

회의명	1차 제안서 발표 및 보완 항목 회의		
일 시	2월 19일	장 소	비대면
의 안	발표 분담, 제안서 필수 항목, MVP 범위 및 위험 요소 점검		
토의내용	1차 발표는 하대영, 노가영, 함서현 순서로 분담하고, 전체 발표 흐름은 문제 정의, 목표 시스템, 기존 시스템 조사, 기능 명세, 기술 스택, 역할 분담, 개발 환경과 일정 순서로 구성하기로 하였다. 제안서에는 기존 헬스 앱의 한계, 기록 앱과 실행 연결 앱의 차이, 타겟 사용자, 사용자 시나리오, 입출력/UI/제약 조건을 더 명확히 보강하기로 하였다. 시스템 설계에서는 건강 데이터 처리 방식, 규칙 기반 컨디션 분석, 종목/홈트 추천 구조, 공공시설 데이터 활용 방식, 게이미피케이션 범위, AI 코치 챗봇 범위를 핵심 논의 항목으로 정리하였다. 기술 스택은 React Native, Spring Boot, Python FastAPI, LangChain/LangGraph, PostgreSQL, Redis를 기준으로 정리하고, 건강 정보 정확도, 부상 책임, 공공데이터 오류, LLM 환각은 위험 요소로 명시하기로 하였다. MVP는 건강 데이터 기반 강도 추천, 위치 기반 시설 추천, 홈트 루틴 추천을 우선 구현하고, AI 챗봇과 고도화 기능은 이후 확장 범위로 두기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유
	하대영 노가영 함서현	하대영	-

			
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

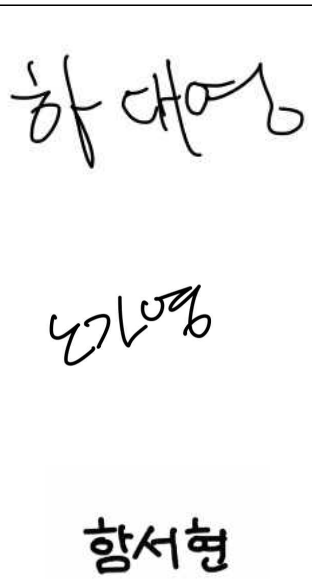
회의명	RAG 데이터 조사 및 모바일 UI 방향 회의		
일 시	2월 28일	장 소	비대면
의 안	운동 RAG 데이터, 건강 데이터 연동 방식, UI 벤치마킹 검토		
토의내용	RAG에는 운동 방법, 주의사항, 운동 지침, 운동 영상/가이드 데이터가 필요하다고 정리하였다. 운동 추천 자체는 규칙과 LLM으로 가능하지만, 세부 동작 설명과 안전 안내는 RAG로 보강하기로 하였다. 건강 데이터 연동은 Android Health API와 Samsung Health SDK의 데이터 구조, 필드, 동기화 방식, 기기 지원 범위를 비교 조사하기로 하였다. 데이터 출처가 웨어러블인지 휴대폰인지 구분 가능한지도 확인하기로 하였다. 공공 운동 장소, 날씨, 미세먼지 API를 각각 조사해 추천 로직에 결합하고, Kaggle 생체 데이터와 FitLife/NHANES/운동 가이드라인 자료는 프로필·상태 분석·운동 추천 근거로 참고하기로 하였다. Planfit 앱을 설치해 추천 화면, 운동 상세, 루틴 UI를 벤치마킹하고, 와이어프레임은 모바일 중심으로 다시 정리하기로 하였다. 추천 화면은 퀘스트와 연결하고, 8bit 이미지 등 게임적 요소를 도입해 게이미피케이션 분위기를 강화하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

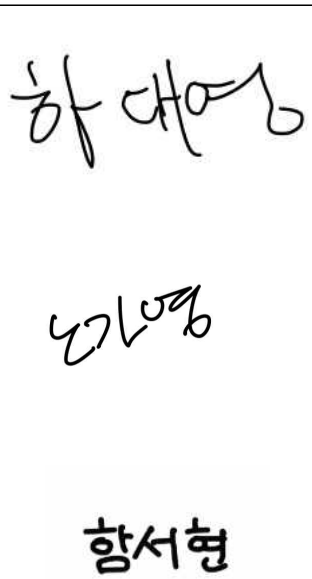
회의명	주제 방향 전환 및 수행 검증 방식 회의		
일 시	3월 8일	장 소	비대면
의 안	웨어러블 기반 운동 추천에서 루틴·퀘스트형 습관 형성 서비스로 전환 검토		
토의내용	기존의 건강 데이터 기반 운동 추천은 의학적 근거와 검증 부담이 크다고 판단하고, 추천/처방보다 운동 수행 유도와 퀘스트형 게이미피케이션에 초점을 맞추기로 하였다. 서비스 방향은 초보자를 위한 운동 습관 형성 루틴 앱으로 재정의하였다. 루틴은 주간 계획, 퀘스트는 오늘 수행할 실행 단위로 역할을 나누기로 하였다. 웨어러블 데이터는 추천 근거가 아니라 걸음 수, 운동 시간, 심박 구간 등을 활용한 수행 검증과 난이도 조절 보조 수단으로 사용하기로 하였다. 운동 루틴은 전문적인 부위분할이 아니라 걷기, 홈트, 수영, 스트레칭 등 초보자용 생활 운동 계획 중심으로 구성하기로 하였다. 웨어러블이 없는 사용자를 위해 GPS 시작/종료 체크 또는 오운완 사진 기반 AI 검증을 후보로 두고, 어떤 방식이 적절한지는 교수님께 확인하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

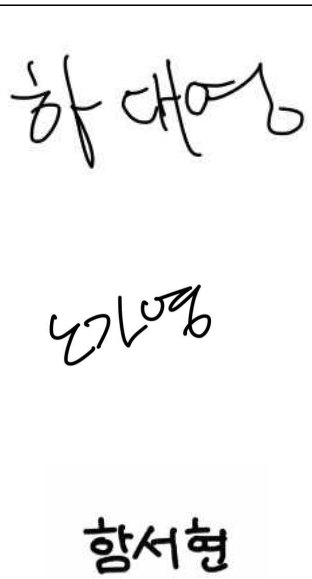
회의명	최종 주제명 및 도메인 구조 재정의 회의		
일 시	3월 14일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	기존 운동 루틴/퀘스트 방향을 캐릭터 챗 기반 게이미피케이션 서비스로 구체화하였다. 최종 주제는 AI 캐릭터 챗 기반 운동 퀘스트 게이미피케이션 서비스로 정리하고, 캐릭터 챗, 운동 퀘스트, 보상, 캐릭터 성장, 경쟁 요소를 핵심 구조로 잡았다. 전체 흐름은 루틴 설정, 캐릭터의 퀘스트 제시, 운동 수행, 보상 획득, 캐릭터 성장 및 경쟁으로 정의하였다. 도메인은 User, Routine, Quest, Scenario/Character, Avatar, Reward, Economy, Competition으로 나누고 각 도메인의 주요 유스케이스를 정리하였다. AI 구현은 LangGraph 기반 Agent workflow와 Story FSM, Pydantic 구조화 출력을 검토하기로 하였으며, 캐릭터 챗이 실제 채팅인지 선택지형 스토리인지와 퀘스트 삽입 방식을 예시 시나리오로 명확히 해야 한다는 의견이 나왔다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

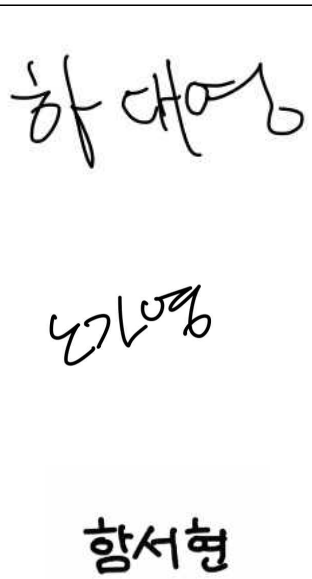
회의명	DDD 도메인 설계 및 역할별 진행 상황 점검 회의		
일 시	3월 27일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	각자 진행 상황과 남은 작업을 공유하였다. 함서현은 팔로잉/팔로워 삭제, 보유중/사용중 상태 변경, 세계관 랭킹 화면, GitHub 로고, Auth 화면, toggle test 등 프론트 작업을 진행하였다. 하대영은 뱃지 지급 정책, DB 통합 v1 초안과 DDL, Tailscale/서버 이용 가이드, 설계명세서 내 백엔드 파트를 담당하기로 하였다. 노가영은 레벨업 정책, 데이터베이스 설계, 다이어그램 작성과 AI/스토리 관련 설계를 맡기로 하였다. 개발 환경은 Android Studio, React Native, TypeScript를 기준으로 정리하였다. 또한 Story, Character, Scenario, Branch, Quest, Progress, Session 등 DDD 기반 핵심 클래스와 관계를 PlantUML로 정리하고, 스토리 진행과 퀘스트 생성이 연결되는 구조를 검토하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

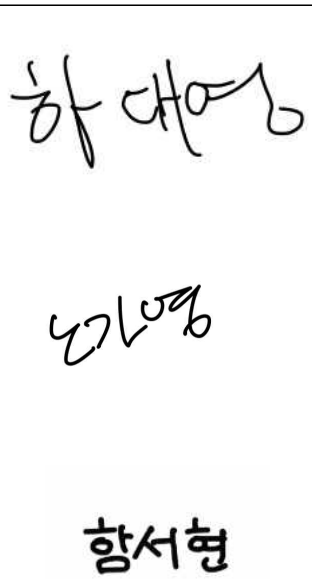
회의명	개발 일정 및 중간 발표 시연 시나리오 회의		
일 시	4월 7일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	정기 회의는 매주 화요일 19시 30분에 진행하기로 정하였다. 프로젝트 협업을 위해 Git 저장소를 개설하고, 기본 브랜치 관리 방식과 작업 공유 방식을 정리하기로 하였다. 백엔드 개발 환경에서는 PostgreSQL을 기본 DB로 사용하고, CI/CD 테스트를 통해 빌드와 테스트가 자동으로 수행되는 구조를 검토하기로 하였다. 중간 발표에서는 전체 기능을 모두 완성하기보다 핵심 사용자 흐름이 보이도록 시연 시나리오를 구성하기로 하였다. 예상 시나리오는 로그인 후 운동 경험과 기본 루틴을 더미 데이터로 설정하고, 사용자가 컨디션을 입력한 뒤 루틴 기반 퀘스트를 조회하고, 캐릭터/세계관 화면에서 스토리를 시작한 다음 퀘스트를 완료하여 보상과 스토리 진행이 이어지는 흐름으로 잡았다. 이를 통해 서비스의 핵심 루프인 로그인, 루틴, 컨디션 입력, 퀘스트 생성, 스토리 진행, 퀘스트 완료를 발표에서 보여주기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

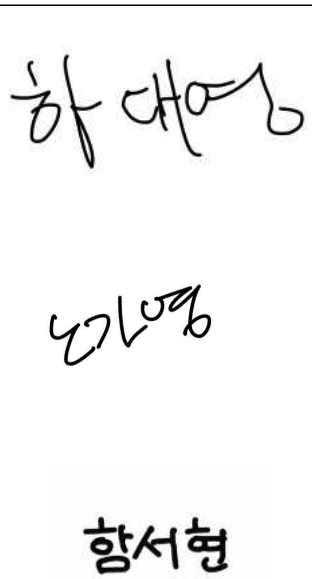
회의명	루틴 기반 퀘스트 생성 및 홈/랭킹 API 설계 회의		
일 시	4월 30일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	온보딩 단계에서 사용자의 운동 경험, 기존 운동 여부, 선호 운동을 미리 입력받고 이를 바탕으로 기본 루틴 1~2개를 추천하는 흐름을 정리하였다. 추천 루틴을 선택하면 사용자 루틴으로 복사되어 활성화되고, 활성 루틴의 요일별 세션을 기준으로 오늘의 루틴과 퀘스트가 생성되도록 설계하기로 하였다. 퀘스트는 루틴 전체를 그대로 수행시키는 방식이 아니라 해당 세션 안의 일부 운동을 자연스럽게 추출해 생성하는 방식이 적절하다고 보았다. 루틴이 없는 날에는 가벼운 퀘스트를 제공하고, 화면 명칭도 오늘의 퀘스트보다 오늘의 루틴 중심으로 조정하기로 하였다. 또한 운동 즐겨찾기, 내 운동 루틴 만들기, 카테고리 포함 즐겨찾기 조회 API, 퀘스트 완료 흐름을 점검하였다. 홈 화면은 슬라이드, 세계관 랭킹 요약, 이번 주 랭킹을 각각 별도 API로 내려주고 프론트에서 조립하는 방식으로 정리하였다. 세계관 랭킹 전체보기는 카드 클릭과 무한 스크롤을 지원하며, 카테고리, 현재 사용자 수, 이미지, 대표 캐릭터 이름 등을 포함해 50개 단위로 조회하도록 계획하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

회의명	중간 발표 피드백 반영 및 기능 안정화 회의		
일 시	5월 13일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	중간 발표 이후 받은 피드백을 기준으로 최종 구현 범위를 재정리하였다. 발표에서 핵심 흐름은 전달되었으나 루틴, 퀘스트, 캐릭터 스토리, 보상 사이의 연결이 더 명확해야 한다는 의견이 있어 사용자 시나리오를 로그인, 루틴 활성화, 컨디션 입력, 오늘의 루틴 조회, 퀘스트 완료, 보상 지급, 스토리 진행 순서로 고정하기로 하였다. 루틴 기반 퀘스트 생성은 활성 루틴의 요일별 세션을 기준으로 처리하고, 퀘스트 완료 직후 경험치와 뱃지가 지급되는 흐름을 우선 구현하기로 하였다. 또한 API 응답 형식, 더미 데이터, 화면 연결 상태를 점검하고 최종 발표 전까지 버그 수정과 기능 안정화를 우선하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

회의명	최종 발표 및 결과보고서 마무리 회의		
일 시	5월 27일	장 소	비대면
의 안	노가영, 하대영, 함서현		
토의내용	최종 발표 전 전체 산출물과 보고서 보완 항목을 점검하였다. 발표 시연은 회원 로그인 후 오늘의 루틴과 퀘스트를 확인하고, 캐릭터 세계관에 진입하여 스토리를 진행한 뒤 퀘스트 완료와 보상 지급을 확인하는 순서로 구성하기로 하였다. 결과보고서에는 구현 기술, 기술 스택, 주요 함수 명세, 소스 코드, 역할 분담, 회의록이 빠짐없이 들어갔는지 확인하기로 하였다. 백엔드는 인증, 루틴, 퀘스트, 보상, 랭킹, 상점 API의 동작을 점검하고, 프론트는 홈 화면, 마이페이지, 세계관 랭킹, 캐릭터 화면의 화면 전환과 데이터 표시를 확인하기로 하였다. AI/스토리 파트는 LangGraph 기반 설계 의도와 캐릭터 챗을 통한 운동 퀘스트 제시 흐름을 발표에서 설명하기로 하였다.		
참석자 서명 및 불참사유	성 명	서 명	불참사유

	하대영 노가영 함서현		-
작성자	소 속 : 컴퓨터공학부 소프트웨어전공 성 명 : 하대영 (서명)		

13. 팀원별 포트폴리오

이 름	함서현
Frontend / App	React Native TypeScript Health Connect 연동 Android 실기기 테스트 UI/UX 구현
Collaboration & Tools	Visual Studio Code Git, GitHub Notion Figma
역할	- 프로젝트 기획 및 UI 구성 - React Native 앱 개발 - 웨어러블 건강 데이터 연동
개발 주요 내역	- 1. React Native 앱 개발 - - Sweet & Sweat의 홈, 퀘스트, 캐릭터, 배틀, 랭킹, 마이페이지 등 모든 화면을 구현함. - - 사용자가 운동 목표와 보상 정보를 쉽게 확인할 수 있도록 UI를 구성함. - 2. 웨어러블 기반 건강 데이터 연동 및 검증 - - Health Connect를 활용하여 걸음 수, 운동 시간 등 건강 데이터를 앱에서 활용할 수 있도록 연동을 진행함. - - 실기기와 웨어러블 기기 연동 과정에서 발생하는 권한 설정, 데이터 수집, 연결 문제를 확인하고 테스트함. - 3. 앱 기능 및 API 연동 테스트 전담 - - 회원가입, 로그인, 퀘스트 조회, 캐릭터 선택, 랭킹 조회, 상점 이용 등 주요 기능의 동작 여부를 사용자 관점에서 테스트함. - React Native 앱과 Spring Boot 서버 간 API 요청·응답 흐름이 정상적으로 이루어지는지 검증함. - - 화면 이동, 버튼 동작, 입력값 처리, 상태 코드, 응답값, 에러 메시지 등을 확인하며 기능 오류와 연동 오류를 점검함. - - 테스트 중 발견한 오류를 팀원들과 공유하고, 수정 후 재검증을 진행하여 - 앱의 안정성과 완성도를 높임. -

이 름	노가영
Backend / AI Server	Python, FastAPI, LangChain/LangGraph, PostgreSQL, SQLAlchemy, OpenAI API REST API 연동, Swagger 기반 API 테스트, Git/GitHub
Collaboration & Tools	VS Code/Cursor, GitHub, Notion, Swagger, PostgreSQL/pg_dump, Tailscale 개발 서버 공유
역할	AI 스토리 서버 개발, LangGraph 기반 스토리 워크플로우 설계 시나리오 생성/플레이 API, 캐릭터 대화·선택지·운동 퀘스트 연동 로직 구현 PostgreSQL 데이터 관리 및 백엔드/프론트 연동 지원
개발 주요 내역	• 1. AI 스토리 생성 및 플레이 API 개발: /stories/generate, /stories/scenarios 등 시나리오 생성·조회 흐름 구현 • 2. LangGraph 기반 스토리 진행: 사용자 입력, 캐릭터 상태, 선택지를 반영해 장면 진행 구조 설계 • 3. 캐릭터 대화/선택지/운동 퀘스트 연동: 스토리 맥락에 맞는 대사와 퀘스트 제시 흐름 구현 • 4. AI 서버 DB 관리: PostgreSQL 데이터 구조, dump 파일, API 응답 형식을 정리해 팀원 연동 지원

이 름	하대영
Backend / Server	Java 17, Spring Boot 3, Spring Security/JWT, PostgreSQL, JPA/QueryDSL Redis, Docker/Tailscale, GitHub Actions, Swagger/API 테스트
Collaboration & Tools	IntelliJ IDEA, Git/GitHub, GitHub Actions, PostgreSQL/psql, Docker, Tailscale, Swagger, Notion
역할	- 팀장, Spring Boot 백엔드 서버 개발, DB 구조 설계 - REST API 명세 및 구현, 프론트엔드/AI 서버 연동 지원 - 개발 서버 및 협업 환경 관리
개발 주요 내역	- 1. 백엔드 API 서버 구현: 회원가입/로그인/JWT, 온보딩, 사용자 정보, 루틴·컨디션·퀘스트 API 개발 - 2. 퀘스트/보상/게임화 기능: 요일별 루틴 세션 기반 퀘스트 생성, 완료 처리, 경험치·재화·배지 지급 흐름 구현 - 3. 상점/배틀/랭킹 도메인: 캐릭터·아이템 보유 상태, 구매, 경쟁 기록과 랭킹 API 구현 - 4. DB 및 서버 운용: PostgreSQL DDL, Docker/Tailscale 개발 서버, CI/CD 테스트와 API 문서화 관리 - 5. AI 서버 연동 지원: 모바일 앱과 AI 서버 사이 요청/응답 구조를 정리하고 스토리·퀘스트 연결 흐름 보완